



بسمه تعالی

آزمایشگاه مخابرات دیجیتال

استاد: دکتر علی الفت

ویرایش ۱/۸ (آزمایشی)

فهرست مطالب آزمایشگاه مخابرات دیجیتال

۳.....	مقدمه: معرفی آزمایشگاه مخابرات دیجیتال
۹.....	آزمایش اول: پردازش سیگنال دیجیتال با نرم افزار MATLAB
۹.....	آزمایش دوم: معادل باند پایه و میان گذر
۱۴.....	آزمایش سوم: معرفی مدولاسیون دیجیتال خطی
۲۶.....	آزمایش چهارم: کاوش در مدولاسیون دیجیتال خطی
۳۴.....	آزمایش پنجم: مدولاسیون FSK همدوس
۴۳.....	آزمایش ششم: آشکارسازی ناهمدوس
۵۱.....	آزمایش هفتم: انتقال دیجیتال از درون کانال باند محدود AWGN



مقدمه

معرفی آزمایشگاه مخابرات دیجیتال

رویکرد فعالیت‌های آزمایشگاه مخابرات دیجیتال

به عنوان مقدمه، مطالبی در مورد رویکرد فعالیت‌های آزمایشگاه مخابرات دیجیتال ارائه می‌شود. آزمایشگاه مخابرات دیجیتال در این ترم، شامل ۱۰ جلسه سه ساعته می‌باشد. دانشجو در طول این ۱۰ جلسه با مفاهیم مخابرات دیجیتال، ابزار شبیه‌سازی و یک سخت‌افزار آموزشی این حوزه آشنا می‌شود.

عمده‌ی آزمایشگاه‌هایی که در حوزه‌ی مخابرات سیستم برگزار می‌گردد، بر مبنای شبیه‌سازی است. این امر باعث می‌شود دانشجو با چالش‌های پیاده‌سازی عملی الگوریتم‌ها آشنا نشود. علاوه بر این معمولاً سخت‌افزارهایی که پیاده‌سازی‌های عملی بر روی آن صورت می‌پذیرد، دارای پیچیدگی بالایی است. از این رو می‌بایست مدت زمان قابل توجهی صرف راه‌اندازی و آموزش این سخت‌افزارها شود. این امر باعث می‌شود فرصتی برای آموزش مفاهیم حوزه‌ی مخابرات دیجیتال باقی نماند.

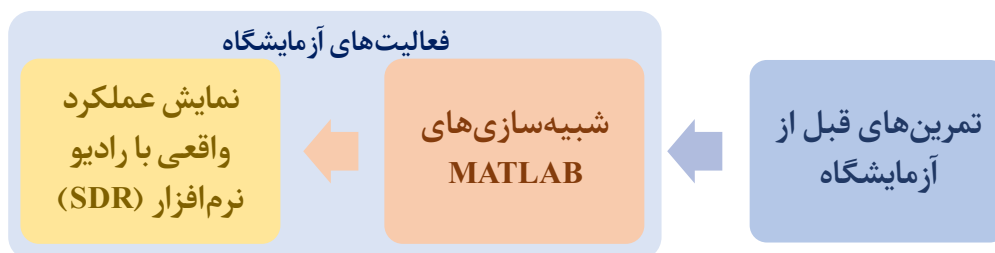
مطلب حائز اهمیت دیگر آن است که دانشجو‌هایی که وارد این آزمایشگاه می‌شوند، تسلط کاملی نسبت به ابزارهای شبیه‌سازی ندارند و مدت زمان قابل توجهی در طول آزمایش‌ها، صرف آموزش این ابزارها می‌شود. بنابراین می‌بایست یک بسته‌ی آموزشی برای تسلط بر روی ابزارهای شبیه‌سازی تهیه شود. به منظور کاهش زمان این آموزش، می‌بایست تمرکز بر روی یک ابزار شبیه‌سازی باشد و راه‌اندازی سخت‌افزار و شبیه‌سازی‌های متکی بر آن با استفاده از همین یک ابزار انجام شود.

با استدلال‌های ذکر شده طراحی این آزمایشگاه مبتنی بر نرم‌افزار MATLAB و تنها اختصاص به بخش کدنویسی آن دارد. به علت محدودیت زمان، امکان آموزش بخش Simulink این نرم‌افزار نیز وجود ندارد. سخت‌افزاری که برای بخش عملی آزمایشگاه مورد استفاده قرار می‌گیرد، رادیو نرم‌افزار ADALM-PLUTO شرکت Analog Devices می‌باشد که در شکل ۱ نشان داده شده است. این رادیو نرم‌افزار را می‌توان به راحتی با برنامه‌ی MATLAB کنترل نمود و از آن برای دریافت و ارسال سیگنال استفاده نمود و یک سامانه‌ی کامل مخابرات دیجیتال را پیاده‌سازی کرد.



شکل ۱ رادیو نرم‌افزار ADALM-PLUTO

فعالیت‌های این آزمایشگاه شامل سه بخش تمرین‌های قبل از آزمایشگاه، شبیه‌سازی درون آزمایشگاه و نمایش عملکرد واقعی با رادیو نرم‌افزار می‌باشد. این روند در شکل ۲ نشان داده شده است.



شکل ۲ روند فعالیت‌های آزمایشگاه مخابرات دیجیتال

در این روند به صورت تدریجی پایه‌های هر مرحله در مرحله‌ی قبل چیده می‌شود تا آزمایش با موفقیت و با بیشترین بازده انجام شود. با این رویکرد، روند پیشرفت آزمایش‌ها با سرعت بالاتری انجام شده و دانشجو بر روی مطالب و مفاهیم مطرح شده در هر آزمایش تسلط بیشتری کسب خواهد نمود.

به منظور تسلط بر مباحث تئوری ارائه شده در درس‌های گذشته نظیر مخابرات دیجیتال و آمادگی برای شبیه‌سازی آزمایش‌ها، تمرین‌هایی به عنوان فعالیت قبل از کلاس در نظر گرفته شده است. دانشجویان با انجام تمرین‌ها قبل از آزمایشگاه، با مفاهیم کلیدی هر آزمایش آشنا می‌شوند. زمانی که پایه‌های تئوری هر آزمایش بنا شد، دانشجویان موظفند شبیه‌سازی‌های مربوط به طراحی و پیاده‌سازی اجزای یک سامانه‌ی مخابرات دیجیتال را انجام دهند و نتیجه‌ی کار را در محیط شبیه‌سازی نرم‌افزار مهندسی MATLAB ارزیابی و مشاهده نمایند. منطق این شبیه‌سازی‌ها آن است که دانشجو رفتار عملکردی اجزای یک سامانه‌ی مخابرات دیجیتال را در یک محیط کنترل شده مشاهده نماید. در نتیجه دانشجو آمادگی کافی جهت ورود به دنیای واقعی با عوامل غیر ایده‌آل فراوان را کسب می‌نماید. زمانی که مجموعه‌ی مهارت‌های دانشجویان با آمادگی تئوریک و شبیه‌سازی تقویت گردید، آمادگی لازم برای ورود به پیاده‌سازی اجزای سامانه‌ی مخابرات دیجیتال در یک محیط بی‌سیم واقعی به دست می‌آید. دانشجو در این مرحله با استفاده از ابزارهایی که نرم‌افزار MATLAB در اختیار می‌گذارد، عملکرد رادیو نرم‌افزار را مشاهده می‌نماید.

شیوه‌ی ارزیابی آزمایشگاه

در ادامه سهم هر یک از فعالیت‌های آزمایشگاه در نمره‌ی پایانی آمده است. دقت نمایید که همه‌ی مراحل به صورت انفرادی می‌باشد.

- تمرین‌های قبل از آزمایش: ۶ نمره
- انجام آزمایش‌ها: ۸ نمره
- گزارش کار آزمایشگاه: ۲ نمره
- پروژه‌ی پایانی یا آزمون: ۴ نمره
- حضور و غیاب: هر جلسه غیبت کسر ۱ نمره

در این ترم دستورکار آزمایشگاه در اختیار دانشجو قرار می‌گیرد. در روز آزمایشگاه توضیحاتی در مورد آزمایش ارائه می‌شود. هر کدام از گروه‌ها در طول سه ساعت آزمایشگاه با همیاران آموزشی آزمایشگاه در ارتباط هستند و به کمک آن‌ها کدهای خود را رفع اشکال می‌نمایند. در پایان روز ارائه‌ی درس، تمرین‌های قبل از آزمایش برای هفته‌ی آینده بارگزاری شده و دانشجویان باید تا یک روز قبل از شروع کلاس بعد جواب‌های آن را در سایت بارگزاری نمایند.



آزمایش اول

پردازش سیگنال دیجیتال با نرم افزار MATLAB

اهداف آزمایش

پس از آشنایی با امکاناتی که نرم افزار MATLAB در اختیار قرار می دهد، در این پیش نیاز بناست تا این امکانات را در حوزه پردازش سیگنال های دیجیتال و سامانه های مخابراتی به کار ببندیم. در طول آزمایش های بعدی مکرر با سیگنال های گسسته زمان سر و کار داریم. از این رو دانشجو می بایست کار با این سیگنال های گسسته زمان، فرایندها و محاسباتی که بر روی این سیگنال ها صورت می پذیرد را به منزله ی الفبا بیاموزد و بر روی آن ها مسلط باشد. به عبارت دیگر در آزمایش های بعد مکرر با مفاهیم، ابزارها و روش هایی از حوزه ی پردازش سیگنال و سامانه های مخابراتی روبه رو هستیم. از این رو می بایست این موارد در ابتدای این آزمایشگاه آموخته شود تا بتوان اصول سامانه های مخابرات دیجیتال را با سرعت و کیفیت بهتری فراگرفت.

در انتهای این آزمایش دانشجو می بایست:

- بتواند سیگنال های گسسته زمان مختلف را به همراه اندیس گذاری زمانی تولید نماید.
- بتواند عملیات مختلف نظیر جمع، ضرب، مقیاس کردن، جابه جایی زمانی و ... یک یا چند سیگنال را انجام دهد.
- خروجی عبور سیگنال از یک سامانه ی خطی تغییرناپذیر با زمان را با عمل کانولوشن به دست آورد.
- کار با معادلات تفاضلی و پیاده سازی یک سیستم گسسته زمان با استفاده از این معادلات را در نرم افزار MATLAB آشنا باشد.
- بتواند طیف فرکانسی سیگنال های گسسته زمان را به دست آورد.
- تحلیل سامانه های خطی تغییرناپذیر با زمان را در حوزه فرکانس انجام دهد.
- بتواند به راحتی توان و انرژی سیگنال های گسسته زمان را به دست آورد.
- متغیرهای تصادفی را تولید نماید و آمارگان های آن ها را محاسبه نماید.
- بتواند چگالی طیف توان یک فرآیند تصادفی را به دست آورد.
- به مفهوم همبستگی و کاربردهای آن مسلط باشد.

ابزارهای مورد نیاز

رایانه

نرم افزار MATLAB R2020b

شرح آزمایش

آزمایش ۱-۱: تولید سیگنال دیجیتال و عملیات بر روی آن

۱. **دنباله‌های تصادفی:** دنباله‌های تصادفی زیر را تولید نمایید. هم‌چنین با استفاده از دستور **histogram**، هیستوگرام این دنباله را که بر اساس تابع چگالی احتمال، به‌هنگار^۱ شده، رسم نمایید. تعداد میله‌های نمودار برابر ۱۰۰ باشد. برای محاسبه‌ی دنباله‌های زیر می‌توان از دستورهای **rand** و **randn** استفاده نمود.
 - ا. $x_1[n]$ یک دنباله‌ی تصادفی با نمونه‌های مستقل و با توزیع گاوسی با میانگین ۱۰ و واریانس ۱۰ می‌باشد. ۱۰۰۰۰ نمونه‌ی این دنباله را تولید نمایید. (راهنمایی: $(X \sim \mathcal{N}(0,1) \rightarrow \sigma X + m \sim \mathcal{N}(m, \sigma^2))$)
 - ب. $x_3[n] = x_2[n] + x_2[n-1]$ که در آن $x_2[n]$ دنباله‌ای با نمونه‌های مستقل و با توزیع یکنواخت در بازه‌ی $[0,2]$ می‌باشد. ۱۰۰۰۰۰ نمونه‌ی این دنباله را تولید نمایید. چرا شکل نمودار هیستوگرام بدین شکل است؟ (راهنمایی: $(X \sim U(0,1) \rightarrow aX + b \sim U(b, a+b))$)
 - ت. $x_4[n] = \sum_{k=1}^4 y_k[n]$ که هر یک از دنباله‌های $y_k[n]$ مستقل از بقیه و نمونه‌های آن به صورت یکنواخت بین $[-0.5, 0.5]$ توزیع شده است. ۱۰۰۰۰۰ نمونه‌ی این دنباله را تولید نمایید. نمودار هیستوگرام به چه شکلی در آمده و علت آن چیست؟

۲. **کاهش نرخ نمونه‌برداری:** عمل کاهش نرخ نمونه‌برداری یک سیگنال به صورت $y[n] = x[nM]$ تعریف می‌شود. در این‌جا نرخ نمونه‌برداری سیگنال با ضریب صحیح M کاهش می‌یابد. به عنوان مثال اگر نرخ نمونه‌برداری دنباله‌ی $x[n] = \{-2; 4; 3; -6; 5; -1; 8\}$ را با ضریب ۲ کاهش دهیم به دنباله‌ی $y[n] = \{-2; 3; 5; 8\}$ دست پیدا می‌نماییم.
 - ا. در نرم‌افزار MATLAB تابع **downsmpl** را به صورت زیر بنویسید تا عملیات فوق را انجام دهد.
function [y] = downsmpl(x,M)
 - ب. تابع $x[n] = \sin[0.125\pi n]$ را در بازه‌ی $-50 \leq n \leq 50$ تولید نمایید. نرخ نمونه‌برداری $x[n]$ را با ضریب ۴ کاهش دهید و $y[n]$ را تولید نمایید. هر دو دنباله‌ی $x[n]$ و $y[n]$ را با انتخاب مناسب نقاط محور افقی با استفاده از دستور **stem** بر روی هم رسم نمایید.
 - ت. عملیات فوق را برای تابع $x[n] = \sin[0.5\pi n]$ و در بازه‌ی $-50 \leq n \leq 50$ تکرار نمایید.

۳. **افزایش نرخ نمونه‌برداری:** عمل افزایش نرخ نمونه‌برداری یک سیگنال به صورت $y[n] = x[n/N]$ تعریف می‌شود. در این‌جا نرخ نمونه‌برداری سیگنال با ضریب صحیح N افزایش می‌یابد. به عنوان مثال اگر نرخ نمونه‌برداری دنباله‌ی $x[n] = \{-2; 3; 5; 8\}$ را با ضریب ۲ افزایش دهیم به دنباله‌ی $y[n] = \{-2; 0; 3; 0; 5; 0; 8\}$ دست پیدا می‌نماییم.
 - ا. در نرم‌افزار MATLAB تابع **upsmpl** را به صورت زیر بنویسید تا عملیات فوق را انجام دهد.
function [y] = upsmpl(x,N)
 - ب. با استفاده از مثال موجود در صورت سوال عملکرد تابع را ارزیابی نمایید.

آزمایش ۱-۲: عبور سیگنال گسسته از یک سیستم

۱. **حل معادله‌ی تفاضلی:** یک سیستم خطی و تغییر ناپذیر با زمان به صورت معادله‌ی تفاضلی زیر تعریف می‌شود.
 - ا. بدون استفاده از دستورهای **filter** و **impz** پاسخ ضربه‌ی سیستم فوق را در بازه‌ی $0 \leq n \leq 100$ رسم نمایید. سپس با استفاده از دستورهای **filter** و **impz** محاسبات خود را صحت‌سنجی نمایید.
 - ب. اگر ورودی سیستم به صورت $x[n] = (5 + 3 \cos[0.2\pi n] + 4 \sin[0.6\pi n])u[n]$ باشد، پاسخ خروجی $y[n]$ در بازه‌ی $0 \leq n \leq 200$ را بدون استفاده از دستور **filter** تعیین نمایید.

^۱ Normalized

۲. تابع همبستگی و کاربرد آن:

- ا. تابعی بنویسید که با استفاده از فرم مختلط فرمول مستقیم همبستگی، همبستگی دو دنباله‌ی x و y را محاسبه نماید. عنوان تابع به صورت `function [r] = corrM(x, y)` باشد. در این جا طول خروجی برابر با طول سیگنال بلندتر منهای طول سیگنال کوچک‌تر به علاوه‌ی ۱ می‌باشد. به عبارتی تنها نمونه‌هایی که اشتراک دو سیگنال برابر با طول سیگنال کوچک‌تر است، در خروجی ظاهر شود.
- ب. تعدادی بسته‌ی داده با یک مدولاسیون دیجیتال از یک فرستنده به سمت یک گیرنده ارسال می‌شود. ابتدای هر بسته یک هدر وجود دارد تا بتوان به وسیله‌ی آن هدر ابتدای بسته را پیدا کرد. سیگنال ارسالی در هوا با سرعت نور ($c = 3 \times 10^8 \text{ m/s}$) سرعت انتشار نور) منتشر می‌شود و به گیرنده می‌رسد. فرض کنید گیرنده لحظه‌ی شروع ارسال را می‌داند و از آن جا شروع به دریافت سیگنال می‌نماید (تحقق عملی دشواری دارد). لحظه‌ای که سیگنال دریافت می‌شود نسبت به لحظه‌ی ارسال بسته با T مشخص می‌شود و فاصله‌ی گیرنده و فرستنده نیز با $R = cT$ تعیین می‌شود. فایل `PacketsAndHeader.mat` شامل سیگنال دریافتی در گیرنده و هدر آن می‌باشد. با استفاده از تابع همبستگی و با فرض ثابت بودن فاصله‌ی گیرنده و فرستنده، این فاصله و طول زمانی هر بسته را به دست آورید. اگر بدانید تعداد نمونه‌های زمانی هر سمبل در این فایل ارسالی برابر با ۸ است، تعداد سمبل‌ها رو نیز حساب کنید. نمودار بخش حقیقی و موهومی مربوط به داده‌های هر دو متغیر موجود در فایل ذکر شده را نیز رسم نمایید. فرکانس نمونه‌برداری مبدل آنالوگ به دیجیتال گیرنده و فرستنده برابر با 10MHz بوده است (راهنمایی: می‌توانید از تابع `load` برای بازکردن فایل‌های داده شده استفاده کنید).

آزمایش ۳-۱: تحلیل حوزه‌ی فرکانس

۱. محاسبه‌ی طیف سیگنال: دستور `fft` در نرم‌افزار MATLAB عمل تبدیل فوریه‌ی گسسته‌ی سریع را انجام می‌دهد. این دستور تنها تعدادی نمونه‌ی زمانی دریافت کرده و به همان اندازه و یا برابر تعداد نقاطی که به دستور گفته می‌شود، نمونه‌ی فرکانسی تولید می‌نماید. فرض کنید فرکانس نمونه‌برداری برابر 250MHz می‌باشد. با این فرکانس نمونه‌برداری، سیگنال $x(nT_s) = x[n] = Ae^{j2\pi f_0 nT_s}$ را تولید نمایید و طیف فرکانسی آن را در بازه‌ی $[-125, 125]\text{MHz}$ را رسم نمایید. (راهنمایی: از کد موجود در بسته‌ی آموزشی ۲ می‌توان استفاده کرد.)

- ا. $f_0 = \frac{250 \times 51}{256} \text{ MHz}$, $A = 2$ و تعداد نقاط FFT برابر ۲۵۶ نقطه است.
- ب. $f_0 = \frac{250 \times 51.5}{256} \text{ MHz}$, $A = 2$ و تعداد نقاط FFT برابر ۲۵۶ نقطه است.
- ت. $f_0 = \frac{250 \times 51}{256} \text{ MHz}$, $A = 2$ و تعداد نقاط FFT برابر ۵۱۲ نقطه است.
- ث. $f_0 = -\frac{250 \times 51}{256} \text{ MHz}$, $A = 2$ و تعداد نقاط FFT برابر ۵۱۲ نقطه است.
- ج. $f_0 = \frac{250 \times 153}{256} \text{ MHz}$, $A = 4$ و تعداد نقاط FFT برابر ۵۱۲ نقطه است.
- ح. علت انتخاب فرکانس‌های فوق را کاملاً درک نمایید. به منظور درک بیشتر در هر مرحله نسبت طول زمانی یک فریم FFT ($N_{\text{FFT}}T_s$) را به دوره تناوب سیگنال ($1/f_0$) را به دست آورید. مفهوم ناشی طیف^۲ را با استفاده از مشاهدات صورت گرفته توضیح دهید.
- خ. اثر تعداد نقاط FFT را بر روی دامنه‌ی طیف مشاهده نمایید و سعی کنید با افزودن ضریب مناسب در کد موجود در بخش تئوری این اثر را خنثی کنید. به عبارتی دامنه‌ی طیف می‌بایست برابر A^2 باشد.
- د. اثر تعداد نقاط FFT بر قابلیت تفکیک فرکانسی چیست. برای این منظور فرکانس f_0 را به مقدار کمی تغییر دهید تا مقدار فرکانس آن از یک خانه‌ی FFT به خانه‌ی بعد منتقل شود.
- ذ. توان سیگنال را در حوزه‌ی زمان محاسبه نمایید و با توان به دست آمده در حوزه‌ی فرکانس مقایسه کنید.

² Spectral Leakage

در عمل به جای گزارش دامنه از توان سیگنال بر حسب dBm استفاده می شود. همان مقدار دسیبل توان در هر میلی وات است:

$$P \text{ وات} \leftrightarrow 10 \log(P) \text{ dBW} \leftrightarrow 10 \log\left(\frac{P}{1\text{mW}}\right) = 10 \log(P) + 30 \text{ dBm}$$

معمولا توان را با فرض مقاومت ۵۰ اهم از روی ولتاژ موثر به دست می آورند. به عنوان مثال برای سیگنال متناوب

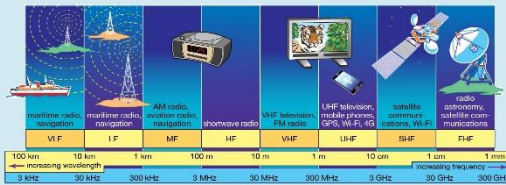
$$x[n] = A \cos[2\pi f_0 n T_s] \text{ dBm به صورت زیر انجام می شود:}$$

$$A \text{ دامنه} \rightarrow \frac{A}{\sqrt{2}} \text{ توان موثر} \rightarrow \frac{A^2}{2R} \text{ watt} \rightarrow 10 \log \frac{A^2}{2R} \text{ dBW} \rightarrow \left(10 \log \frac{A^2}{2R} + 30\right) \text{ dBm}$$

حال با این اطلاعات طیف سیگنال بخش ا و ب را که بر حسب dBm است رسم نمایید. در نرم افزار MATLAB با استفاده از دستور db بسیاری از این تبدیل ها انجام می گیرد که اعمال مقاومت در آن نیازمند دقت است.



- [1] V. K. Ingle and J. G. Proakis, *Digital Signal Processing using MATLAB*. 2010.
- [2] J. G. Proakis, M. Salehi, and G. Bauch, *Contemporary Communication Systems Using MATLAB®*. 2013.
- [3] S. K. Mitra, *Digital signal processing: a computer-based approach*. New Delhi: Mc Graw Hill Education, 2011.
- [4] Ingle, V. K., & Proakis, J. G. (2012). *Digital signal processing using MATLAB*. Stanford: Cengage Learning.
- [5] T. F. Collins, R. Getz, D. Pu, and A. M. Wyglinski, *Software-defined radio for engineers*. Norwood, MA: Artech House, 2018.



آزمایش دوم

معادل باندپایه و میان گذر

اهداف آزمایش

طیف رادیویی به بخشی از طیف الکترومغناطیس در محدوده‌ی فرکانسی 3kHz تا 300GHz گفته می‌شود. از طیف رادیویی به صورت گسترده در سرویس‌های مخابراتی نظیر ارسال سیگنال‌های تلویزیون، رادیو، موبایل، WiFi و سامانه‌های ناوبری و آشکارسازی نظیر رادار، GPS، دیده‌بانی رادیویی، فرستنده‌ها و ... استفاده می‌شود. آزمایش‌هایی که صورت می‌پذیرد کمک می‌کند تا این بخش از طیف مورد کنکاش قرار بگیرد و دانشجو به جست‌جو و شناسایی سیگنال‌های رادیویی در این محدوده‌ی فرکانسی بپردازد. از جمله مفاهیمی که در رادیو نرم‌افزارها و سامانه‌های مخابراتی مورد استفاده قرار می‌گیرد، مفهوم معادل پایین‌گذر و میان‌گذر سیگنال است. دانشجو در این آزمایش با این مفاهیم آشنایی کامل پیدا می‌کند. در انتهای این آزمایش دانشجو می‌بایست:

- 🔧 تسلط کامل بر روی مفاهیم معادل پایین‌گذر و بالاگذر داشته باشد و بتواند این دو معادل را به یکدیگر تبدیل نماید.
- 🔧 با طیف فرکانسی محدوده‌ی 25MHz تا 6GHz آشنا باشد و درک مناسبی از سیگنال‌های موجود در این گستره‌ی فرکانسی داشته باشد.

ابزارهای مورد نیاز

رایانه

نرم‌افزار MATLAB R2020b

- MathWorks DSP System Toolbox
- MathWorks Communications System Toolbox
- MathWorks Signal Processing Toolbox



تئوری

معادل باندپایه‌ی مختلط سیگنال‌های میان‌گذر

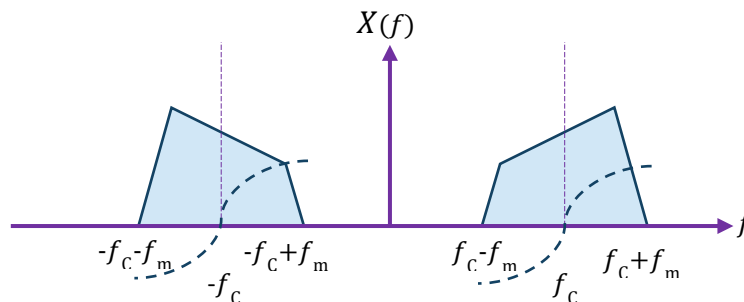
تقریباً همه‌ی سامانه‌های مخابراتی با مدوله کردن یک شکل موج حاوی اطلاعات بر روی یک سیگنال حامل سینوسی پیاده‌سازی می‌شوند. فرکانس حامل سیگنال ارسالی حاوی هیچ اطلاعاتی نیست و در واقع سیگنال مدوله‌کننده‌ی حامل در برگیرنده‌ی اطلاعات است. از این رو به توصیف سیگنال‌های مخابراتی، مستقل از فرکانس حامل نیاز است. در نتیجه‌ی این نیاز، مهندسين سامانه‌های مخابراتی برای سادگی کارها از معادل باندپایه‌ی مختلط^۳ یا معادل پایین‌گذر سیگنال‌های مخابراتی استفاده می‌نمایند. یک سامانه‌ی مخابراتی می‌تواند با بیان باندپایه‌ی مختلط تحلیل شود و معمولاً نیز چنین کاری صورت می‌پذیرد. یکی از مزایای معادل باندپایه‌ی مختلط سادگی است که در آن همه‌ی سیگنال‌ها پایین‌گذر بوده و ایده‌های پایه‌ای مدولاسیون و پردازش سیگنال مخابراتی به سادگی توسعه می‌یابد. علاوه بر این عمده‌ی گیرنده‌هایی که شکل موج‌های دریافتی را به صورت دیجیتال پردازش می‌نمایند، از معادل باندپایه‌ی مختلط برای توسعه‌ی الگوریتم‌های پردازش استفاده می‌نمایند.

گام اول در بسط معادل باندپایه‌ی مختلط، تعریف سیگنال باندپایه است.

تعریف کیفی سیگنال میان‌گذر: سیگنال میان‌گذر $x(t)$ سیگنالی است که مرکز طیف انرژی یک طرفه‌ی آن در فرکانس غیرصفر f_c باشد و دنباله‌ی این طیف تا فرکانس صفر ادامه ندارد.

تعریف ریاضی سیگنال میان‌گذر: سیگنال میان‌گذر $x(t)$ سیگنالی است که **اولاً** حقیقی باشد و **ثانیاً** تبدیل فوری‌ی آن دارای این ویژگی باشد که بتوان f_c و f_m را به نحوی پیدا کرد که $f_c > f_m$ باشد و $\forall f: |f| - f_c > f_m \rightarrow X(f) = 0$

اگر پهنای باند دوطرفه‌ی انتقال سیگنال میان‌گذر برابر با $2f_m$ هرتز باشد، طیف یک طرفه‌ی سیگنال میان‌گذر تنها در بازه‌ی فرکانسی $[f_c - f_m, f_c + f_m]$ غیرصفر می‌باشد. این مسأله بدان معناست که سیگنال‌های میان‌گذر دارای قید $f_m < f_c$ می‌باشند. شکل ۱ نمونه‌ای از یک طیف میان‌گذر را نشان می‌دهد. از آن جا که یک سیگنال میان‌گذر $x_c(t)$ سیگنالی است که به صورت فیزیکی قابل تحقق است، سیگنالی حقیقی بوده و انرژی سیگنال نسبت به فرکانس $f = 0$ متقارن است. اندازه‌ی نسبی B_T و f_c اهمیت نداشته و تنها می‌بایست طیف سیگنال مقدار ناچیزی در حول فرکانس DC داشته باشد. در مخابرات تلفنی این ناحیه‌ی طیف، ناچیز و در حدود 300Hz است در حالی که در مخابرات ماهواره‌ای این مقدار می‌تواند چندین گیگاهرتز باشد.



شکل ۱ پاسخ فرکانسی یک سیگنال حقیقی میان‌گذر

یک سیگنال میان‌گذر دارای بیانی به صورت زیر است:

$$x(t) = x_i(t) \cos 2\pi f_c t - x_q(t) \sin 2\pi f_c t \quad (1)$$

$$= x_a(t) \cos(2\pi f_c t + x_p(t)) \quad (2)$$

در رابطه‌ی فوق f_c فرکانس حامل بوده که در محدوده‌ی $f_c - f_m \leq f_c \leq f_c + f_m$ قرار دارد. معمولاً به سیگنال $x_i(t)$ در معادله‌ی

(۱) مولفه‌ی هم‌فاز^۴ سیگنال و به سیگنال $x_q(t)$ مولفه‌ی غیرهم‌فاز^۵ سیگنال میان‌گذر گفته می‌شود. $x_i(t)$ و $x_q(t)$ سیگنال‌های پایین‌گذر با مقادیر حقیقی هستند که طیف یک طرفه‌ی انرژی آن در فرکانس‌های بالاتر از $2f_m$ دارای مقدار ناچیزی است. توجه کنید که فرکانس مرکزی سیگنال میان‌گذر f_c و فرکانس حامل f_c همواره با یکدیگر یکسان نیستند. در حالی که f_c به صورت تئوری می‌تواند

³ Complex Baseband Representation

⁴ In-Phase

⁵ Quadrature



زنجیره‌ای از مقادیر را اختیار کند، مقدار بدیهی f_c می‌تواند منجر به ساده‌ترین بیان شود. برای سیگنال حامل معمولاً یک عبارت کسینوسی در نظر گرفته می‌شود. از این رو مولفه‌ی $x_i(t)$ هم‌فاز با سیگنال حامل است. به صورت مشابه عبارت سینوسی به اندازه‌ی 90° درجه با کسینوس یا عبارت سیگنال حامل اختلاف فاز دارد و از این رو مولفه‌ی $x_q(t)$ با حامل غیرهم‌فاز می‌باشد. معادله‌ی (۱) با عنوان فرم کانونی سیگنال میان گذر شناخته می‌شود. معادله‌ی (۲) صورت دامنه و فاز سیگنال میان گذر است که در آن $x_a(t)$ دامنه‌ی سیگنال و $x_p(t)$ فاز سیگنال است. سیگنال میان گذر دارای دو درجه‌ی آزادی است و بیان‌های کانونی I/Q با دامنه و فاز با یکدیگر معادل هستند. تبدیل بین این دو بیان به صورت زیر می‌باشد:

$$x_a(t) = \sqrt{x_i^2(t) + x_q^2(t)}, \quad x_p(t) = \tan^{-1}(x_q(t), x_i(t)) \quad (3)$$

$$x_i(t) = x_a(t) \cos x_p(t), \quad x_q(t) = x_a(t) \sin x_p(t) \quad (4)$$

باید دقت نمود که تابع تانژانت در معادله‌ی (۳) دارای برد $[-\pi, \pi]$ می‌باشد و به عبارتی علامت هر دو $x_i(t)$ و نسبت $x_i(t)$ و $x_q(t)$ برای محاسبه‌ی تابع مورد نیاز است. جزییات تحلیل طراحی مخابراتی تعیین می‌کند که کدام صورت سیگنال باندپایه راحت‌تر به کار گرفته می‌شود.

یک سیگنال با مقادیر مختلط با نام پوش مختلط به صورت زیر تعریف می‌شود:

$$x_z(t) = \frac{1}{2}x_i(t) + \frac{j}{2}x_q(t) = \frac{x_a(t)}{2}e^{jx_p(t)} \quad (5)$$

سیگنال میان گذر اصلی را می‌توان به صورت زیر از روی پوش مختلط به دست آورد:

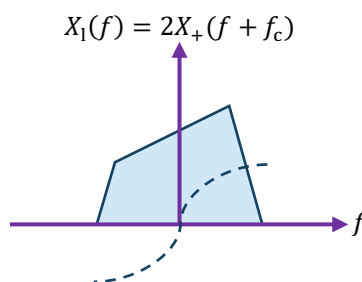
$$x(t) = \Re[2x_i(t)e^{j2\pi f_c t}] \quad (6)$$

از آن جا که تابع نمایی مختلط تنها فرکانس مرکزی را تعیین می‌نماید، سیگنال مختلط $x_i(t)$ همه‌ی اطلاعات موجود در $x(t)$ را در بر دارد. با استفاده از بیان باندپایه‌ی سیگنال‌های میان گذر، تحلیل‌های سامانه‌ی مخابراتی صورت ساده‌تری به خود می‌گیرد.

تبدیل بین سیگنال میان گذر و معادل پایین گذر

در ادامه روشی برای تبدیل بین سیگنال میان گذر و معادل پایین گذر یا سیگنال پوش مختلط آورده شده است. سیگنال معادل پایین گذر که با $x_1(t)$ نمایش داده می‌شود، دارای پاسخ فرکانسی به صورت زیر است که در شکل ۲ نشان داده شده است:

$$X_1(f) = 2X_+(f + f_c) \quad (7)$$

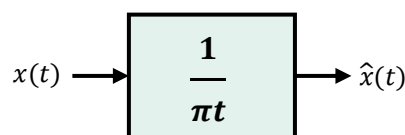


شکل ۲ پاسخ فرکانسی مربوط به معادل پایین گذر سیگنال شکل

می‌توان نشان داد که معادل پایین گذر یک سیگنال میان گذر را می‌توان با استفاده از رابطه‌ی زیر به دست آورد:

$$x_1(t) = (x(t) + j\hat{x}(t))e^{-j2\pi f_c t} \quad (8)$$

در رابطه‌ی فوق $\hat{x}(t)$ ، تبدیل هیلبرت سیگنال $x(t)$ است که در شکل ۳ نشان داده شده است.



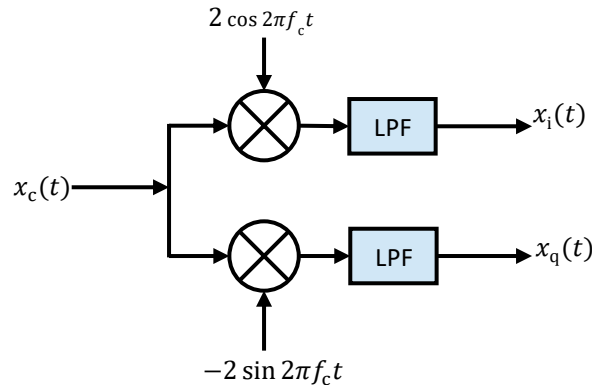
شکل ۳ تبدیل هیلبرت سیگنال $x(t)$

با توجه به رابطه‌ی (۸)، سیگنال $x_1(t)$ یک سیگنال مختلط خواهد بود. همان‌طور که اشاره شد، قسمت حقیقی و مختلط این سیگنال به ترتیب مولفه‌های هم‌فاز و غیرهم‌فاز نامیده شده و با $x_i(t)$ و $x_q(t)$ نشان داده می‌شود. بنابراین می‌توان نوشت:

$$x_1(t) = (x(t) + j\hat{x}(t))e^{-j2\pi f_c t} = x_i(t) + jx_q(t) \quad (9)$$

$$\begin{cases} x_i(t) = x(t) \cos(2\pi f_c t) + \hat{x}(t) \sin(2\pi f_c t) \\ x_q(t) = \hat{x}(t) \cos(2\pi f_c t) - x(t) \sin(2\pi f_c t) \end{cases} \quad (10)$$

از آنجا که فیلتر هیلبرت یک فیلتر غیرعلی است، در عمل برای به دست آوردن $x_i(t)$ و $x_q(t)$ از روشی مشابه شکل ۴ استفاده می‌شود. فیلتر پایین‌گذر می‌بایست سیگنال پوش مختلط را عبور دهد و مولفه‌های دو برابر فرکانس مرکزی را حذف کند.



شکل ۴ به دست آوردن معادل پایین‌گذر بدون استفاده از تبدیل هیلبرت

اگر معادل پایین‌گذر را داشته باشیم، با در نظر گرفتن رابطه‌ی (۹) می‌توان سیگنال میان‌گذر را به دست آورد. معادل میان‌گذر به صورت زیر به دست می‌آید:

$$\begin{cases} x(t) = \Re\{x_1(t)e^{j2\pi f_c t}\} = x_i(t) \cos 2\pi f_c t - x_q(t) \sin 2\pi f_c t \\ \hat{x}(t) = \Im\{x_1(t)e^{j2\pi f_c t}\} = x_q(t) \cos 2\pi f_c t + x_i(t) \sin 2\pi f_c t \end{cases} \quad (11)$$

چگالی طیف توان فرآیند تصادفی ایستاد

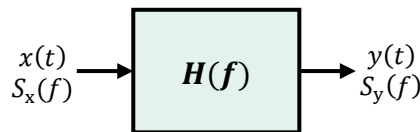
فرآیند $x(t)$ ایستاد به مفهوم وسیع گفته می‌شود اگر میانگین فرآیند $\mathbb{E}[x(t)] = m_x$ مستقل از زمان و تابع خودهمبستگی $R(t + \tau, t) = \mathbb{E}[x(t)x(t + \tau)] = R_x(\tau)$ تنها تابعی از تأخیر τ باشد.

برای فرآیند ایستاد با تابع خودهمبستگی $R_x(\tau)$ ، چگالی طیف توان به صورت زیر محاسبه می‌شود.

$$S_x(f) = \mathcal{F}[R_x(\tau)] \quad (12)$$

یکی از کارهایی که در مواجهه با سیستم‌های خطی تغییر ناپذیر با زمان مورد استفاده قرار می‌گیرد، محاسبه طیف سیگنال خروجی از روی طیف سیگنال ورودی است. این رابطه در زیر آمده است.

$$S_y(f) = S_x(f)|H(f)|^2 \quad (13)$$



شکل ۲ عبور یک فرآیند از سیستم خطی تغییر ناپذیر با زمان

شرح آزمایش



آزمایش ۱-۲: معادل پایین‌گذر و میان‌گذر سیگنال

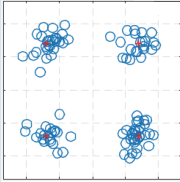
۱. تولید فرآیند میان‌گذر و پایین‌گذر: در صورتی که یک فرآیند تمام‌گذر از یک فیلتر پایین‌گذر عبور کند، یک فرآیند پایین‌گذر حاصل می‌شود. در این جا می‌خواهیم یک فرآیند پایین‌گذر را به معادل میان‌گذر تبدیل نماییم و مجدداً آن را به معادل پایین‌گذر تبدیل کنیم. کلیه‌ی طیف‌ها باید بر حسب dBm باشند و برچسب‌گذاری محور فرکانس به درستی انجام و بر حسب MHz بیان شود.

ا. تولید فرآیند پایین‌گذر: دنباله‌ی تصادفی مختلط با ۴۰۹۶ نمونه‌ی مستقل و با توزیع گاوسی مختلط با میانگین صفر و واریانس ۱ تولید نماییم. با استفاده از ابزار طراحی فیلتر نرم‌افزار MATLAB (دستور **filterDesigner** را در پنجره‌ی دستور وارد نمایید و ضرایب فیلتر را به workspace ارسال نمایید) یک فیلتر پایین‌گذر FIR از نوع Equiripple و با پارامترهای فیلتری $f_s = 28.8\text{MHz}$, $f_{\text{pass}} = 1.4\text{MHz}$, $f_{\text{stop}} = 2\text{MHz}$, $A_{\text{pass}} = 0.5\text{dB}$ و $A_{\text{stop}} = 70\text{dB}$ طراحی نمایید. پس از به دست آوردن ضرایب فیلتر با استفاده از عمل کانولوشن خروجی این فیلتر را برای ورودی تولید شده، به دست آورید. طیف ۴۰۹۶ نقطه‌ای فرآیند تمام‌گذر ورودی و فرآیند پایین‌گذر خروجی را به دست آورید و در یک شکل با دو زیرنمودار رسم نمایید. توان فرآیند را با استفاده از نمونه‌های زمانی قبل و بعد فیلتر شدن به دست آورده و ارتباط آن را با پهنای باند فیلتر به دست آورید (راهنمایی: خروجی عمل کانولوشن تعداد نقاط بیشتری دارد و می‌بایست این تعداد نقاط را برابر با ۴۰۹۶ نقطه نمود. برای به دست آوردن پهنای باند فیلتر از دستور **noisebw** نرم‌افزار MATLAB استفاده نمایید).

ب. تولید فرآیند میان‌گذر معادل یک فرآیند پایین‌گذر: حال با استفاده از مباحثی که در بخش تئوری مطرح شد، فرآیند پایین‌گذر تولید شده را به یک فرآیند میان‌گذر بر روی فرکانس 3.57MHz منتقل نماییم. توان هر سه فرآیند را مشابه قبل به دست آورید. آیا توان فرآیند معادل پایین‌گذر با میان‌گذر یکی است؟ علت را شرح دهید. حال طیف ۴۰۹۶ نقطه‌ای هر سه فرآیند را در یک شکل با سه زیرنمودار رسم نمایید.

ت. تولید فرآیند پایین‌گذر از یک سیگنال میان‌گذر: فرآیند تولید شده در بخش ب را دوباره به معادل پایین‌گذر خود تبدیل نموده و خروجی را از سه فیلتر پایین‌گذر مختلف با پهنای باند 0.7MHz، 1.4MHz و 2.8MHz عبور دهید. طیف فرآیند خروجی این سه فیلتر را با طیف بخش ا مقایسه نمایید و در هر مورد توان خروجی زمانی هر فیلتر را به دست آورید و علت تفاوت‌ها را شرح دهید. فیلترها را با استفاده از دستور **fir1** تولید نموده و مرتبه‌ی فیلتر را برابر با ۱۲۰ در نظر بگیرید. (فرکانس نمونه‌برداری همان $f_s = 28.8\text{MHz}$ بوده و نباید از دستورات مربوط به تبدیل هیلبرت استفاده شود).





آزمایش سوم

معرفی مدولاسیون دیجیتال خطی

اهداف آزمایش

سامانه‌های مخابرات دیجیتال بی‌سیم اطراف ما را فرا گرفته است و ارتباطات صدا و داده در سامانه‌های مخابرات سلولی، WiFi، Bluetooth و سایر استانداردها را فراهم می‌کند. در طی چندین آزمایش آینده بلوک‌های پایه‌ی مخابرات دیجیتال شامل الگوهای مدولاسیون، دریافت سمبل و بیت، عملکرد در حضور نویز، کدگذاری و کدگشایی تقاضایی، تبدیل فرکانس به بالا و پایین دیجیتال و ... ارائه می‌شود. در ادامه جزئیات طراحی گیرنده‌ی دیجیتال شامل همگام‌سازی فرکانس حامل و همگام‌سازی زمانی سمبل‌ها مورد بررسی قرار می‌گیرد. در این آزمایش با الگوهای مدولاسیون دیجیتال خطی و شکل‌دهی پالس آشنا می‌شویم.

در انتهای این آزمایش دانشجو می‌بایست:

- 🔧 منظومه‌های مدولاسیون‌های خطی M-PAM، M-PSK و M-QAM را بشناسد.
- 🔧 بتواند رشته‌های بیت را به سمبل‌های مدولاسیون‌های مختلف در حالت سمبل‌های مختلط و سمبل‌های I/Q تبدیل نماید.
- 🔧 با مفهوم کدگذاری Gray آشنا شود.
- 🔧 بتواند سمبل‌های دریافتی را به رشته‌های بیت تبدیل نماید.
- 🔧 بتواند سیگنال را در یک کانال باندپایه ارسال نماید و اثر نویز را در تبدیل سمبل‌ها به رشته‌های بیت بشناسد.
- 🔧 با طراحی Correlator و فیلتر منطبق آشنا شود.
- 🔧 بتواند گیرنده‌ی حداقل فاصله را طراحی نماید.

ابزارهای مورد نیاز

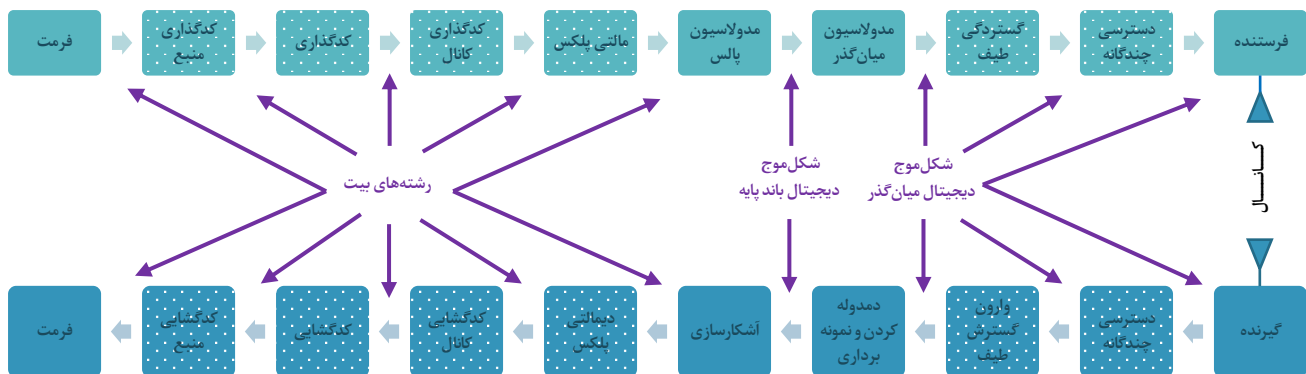
رایانه 

نرم‌افزار MATLAB R2020b 



تئوری

شکل ۱ شمای یک سامانه‌ی مخابرات دیجیتال متداول را به صورت کامل نشان می‌دهد. در این آزمایش و آزمایش بعد سعی داریم که بخش‌های ضروری این سامانه را برای مدولاسیون‌های خطی M-PAM، M-PSK و M-QAM مدل‌سازی کنیم. در ادامه این مدولاسیون‌ها را مرور خواهیم کرد.



شکل ۱ شمای یک سامانه‌ی مخابرات دیجیتال (بخش‌های با پس‌زمینه‌ی نقطه‌دار اختیاری هستند)

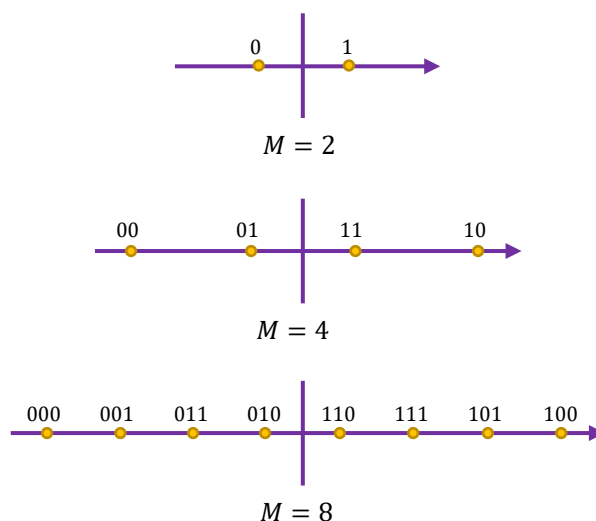
مدولاسیون‌های خطی

مدولاسیون M-PAM (Pulse Amplitude Modulation)

این مدولاسیون از جمله مدولاسیون‌های دیجیتال باند پایه می‌باشد. در این مدولاسیون اطلاعات در توان سیگنال ارسالی ذخیره می‌شود و آشکارسازی آن با تشخیص سطح سیگنال انجام می‌شود.

در این مدولاسیون سیگنال ارسالی به صورت $s_m(t) = A_m p(t)$ است. پالس ارسالی $p(t)$ می‌تواند به طور دلخواه انتخاب شود، اما از آنجا که پهنای باند سیگنال ارسالی به طور کامل به شکل پالس بستگی دارد، تعیین پالس مناسب اهمیت زیادی خواهد داشت. A_m دامنه پالس ارسالی را با توجه به پیغام تنظیم می‌کند. با فرض اینکه هر سیگنال ارسالی حاوی k بیت پیام باشد، می‌توان A_m ها را از مجموعه زیر انتخاب نمود:

$$A_m \in \{\pm 1, \pm 3, \dots, \pm(M-1)\}; \quad M = 2^k$$



شکل ۳ منظومه‌ی سیگنالی مربوط به مدولاسیون M-PAM برای مقادیر مختلف M



مدولاسیون PSK (Phase Shift Keying Modulation)

در این مدولاسیون، سیگنال ارسالی به صورت $s_m(t) = g(t) \cos\left(2\pi f_0 t + \frac{2\pi}{M}(m-1)\right)$ می باشد که f_0 فرکانس حامل خواهد بود. پالس $g(t)$ می تواند به طور دلخواه انتخاب شود. در اینجا نیز همانند مدولاسیون PAM می توان با انتخاب درست این پالس، در پهنای باند صرفه جویی نمود. با فرض اینکه هر سیگنال ارسالی حاوی k بیت پیغام باشد، می توان m را از مجموعه $\{1, 2, \dots, M\}$ انتخاب کرد. در این جا $M = 2^k$ می باشد. با در نظر گرفتن سیگنال ارسالی به صورت فوق، خواهیم داشت:

$$s_m(t) = g(t) \cos(2\pi f_0 t) \cos\left(\frac{2\pi}{M}(m-1)\right) - g(t) \sin(2\pi f_0 t) \sin\left(\frac{2\pi}{M}(m-1)\right)$$

با انتخاب $\Phi_1(t) = \sqrt{\frac{2}{E_g}} g(t) \cos(2\pi f_0 t)$ و $\Phi_2(t) = -\sqrt{\frac{2}{E_g}} g(t) \sin(2\pi f_0 t)$ به عنوان پایه های متعامد فضای سیگنال، خواهیم داشت:

$$s_m(t) = \Phi_1(t) \cos\left(\frac{2\pi}{M}(m-1)\right) \sqrt{\frac{E_g}{2}} + \Phi_2(t) \sin\left(\frac{2\pi}{M}(m-1)\right) \sqrt{\frac{E_g}{2}}$$

در رابطه ی فوق E_g انرژی پالس $g(t)$ می باشد $(\int |g(t)|^2 dt = E_g)$. بدین ترتیب می توان معادل برداری سیگنال $s_m(t)$ را به صورت زیر در نظر گرفت:

$$\underline{s}_m = \sqrt{\frac{E_g}{2}} \begin{bmatrix} \cos\left(\frac{2\pi}{M}(m-1)\right) \\ \sin\left(\frac{2\pi}{M}(m-1)\right) \end{bmatrix}$$

با توجه به $\underline{s}_m$ ، فضای سیگنال این مدولاسیون دارای دو بعد در باند میانی می باشد. معادل باند پایه ی این سیگنال به صورت زیر خواهد بود:

$$s_{l,m}(t) = g(t) e^{j \frac{2\pi}{M}(m-1)}$$

با در نظر گرفتن $\Phi_1(t) = \frac{g(t)}{\sqrt{E_g}}$ به عنوان پایه متعامد فضای سیگنال، داریم:

$$s_{l,m}(t) = \sqrt{E_g} \Phi_1(t) e^{j \frac{2\pi}{M}(m-1)}$$

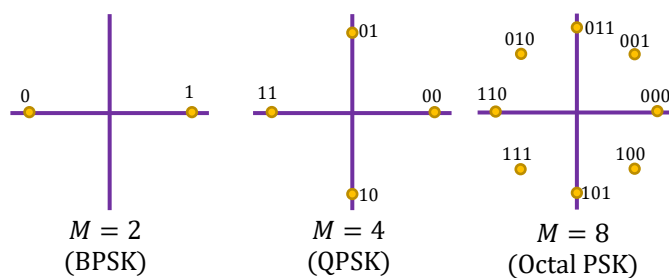
بنابراین در باند پایه، فضای سیگنال تنها یک بعد دارد. به بیان دیگر فضای سیگنال در این مدولاسیون دارای دو بعد حقیقی (باند میانی) یا یک بعد مختلط (باند پایه) خواهد بود. در باند پایه خواهیم داشت:

$$s_{l,m}(t) = g(t) \left(\cos\left(\frac{2\pi}{M}(m-1)\right) + j \sin\left(\frac{2\pi}{M}(m-1)\right) \right)$$

با متحد قرار دادن عبارت فوق با $s_{l,m}(t) = s_i(t) + j s_q(t)$ ، مؤلفه های هم فاز و غیرهم فاز به دست خواهند آمد.

$$\begin{cases} s_i(t) = g(t) \cos\left(\frac{2\pi}{M}(m-1)\right) \\ s_q(t) = g(t) \sin\left(\frac{2\pi}{M}(m-1)\right) \end{cases}$$

همان گونه که ملاحظه می شود، در این مدولاسیون، فاز سیگنال ارسالی، مشخص کننده ی پیغام می باشد.



شکل ۳ منظومه ی مربوط به مدولاسیون MPSK برای مقادیر مختلف M

مدولاسیون QAM (Quadrature Amplitude Modulation)

در این مدولاسیون، ابتدا سیگنال ارسالی را در باند پایه بررسی خواهیم کرد. سیگنال ارسالی مدولاسیون QAM در باند پایه به صورت زیر خواهد بود:

$$S_{l,m}(t) = (A_{m_i} + jA_{m_q})g(t)$$

$g(t)$ پالسی است که برای مدولاسیون انتخاب می شود و همانند دیگر مدولاسیون ها، بر پهنای باند مصرفی تأثیر می گذارد. بعد فضای این سیگنال در باند پایه ۱ می باشد. با در نظر گرفتن $s_{l,m}(t) = s_i(t) + js_q(t)$ ، مؤلفه های in-phase و quadrature به صورت زیر خواهند بود:

$$\begin{cases} s_i(t) = g(t)A_{m_i} \\ s_q(t) = g(t)A_{m_q} \end{cases}$$

معادل باند میانی این سیگنال به صورت زیر به دست می آید:

$$S_m(t) = \text{Re}\{S_{l,m}(t)e^{j2\pi f_0 t}\} = A_{m_i}g(t)\cos(2\pi f_0 t) - A_{m_q}g(t)\sin(2\pi f_0 t)$$

با انتخاب $\Phi_1(t) = \sqrt{\frac{2}{E_g}}g(t)\cos(2\pi f_0 t)$ و $\Phi_2(t) = -\sqrt{\frac{2}{E_g}}g(t)\sin(2\pi f_0 t)$ به عنوان پایه های متعامد فضای سیگنال، خواهیم داشت:

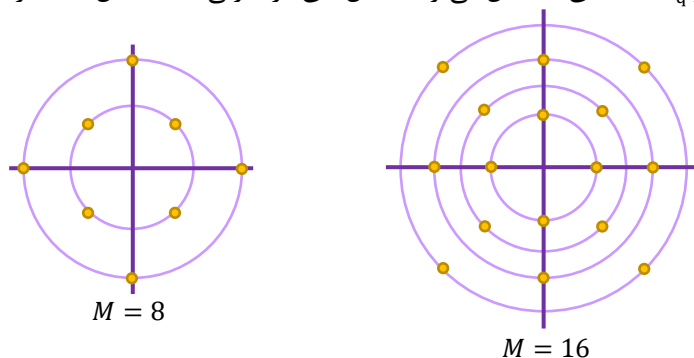
$$S_m(t) = A_{m_i}\sqrt{\frac{E_g}{2}}\Phi_1(t) + A_{m_q}\sqrt{\frac{E_g}{2}}\Phi_2(t)$$

در رابطه ی فوق نیز E_g انرژی پالس $g(t)$ می باشد $(\int |g(t)|^2 dt = E_g)$.

بدین ترتیب می توان معادل برداری سیگنال $s_m(t)$ را به صورت زیر در نظر گرفت:

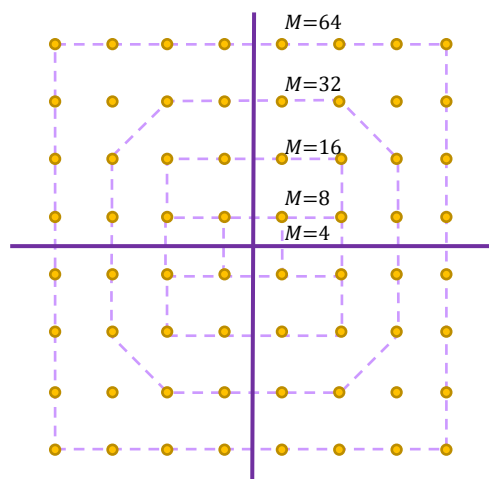
$$\underline{s}_m = \sqrt{\frac{E_g}{2}} \begin{bmatrix} A_{m_i} \\ A_{m_q} \end{bmatrix}$$

به ازای مقادیر مختلف A_{m_i} و A_{m_q} ، فضای سیگنال می تواند شکل های گوناگونی مانند شکل ۴ به خود بگیرد.



شکل ۴ منظومه ی مدولاسیون M-QAM برای مقادیر مختلف M

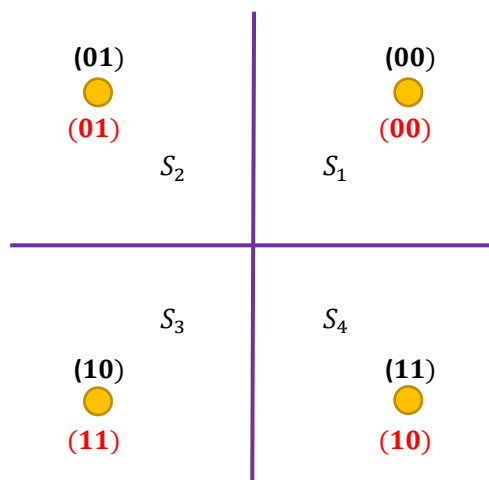
حالت خاص فضای سیگنال به ازای $A_{m_i}, A_{m_q} \in \{\pm 1, \pm 3, \dots, \pm(\sqrt{M}-1)\}$ تشکیل می شود که به QAM مستطیلی معروف است. منظومه ی این مدولاسیون در شکل ۵ نشان داده شده است. از این به بعد، هر جا که درباره مدولاسیون QAM صحبت می شود منظور QAM مستطیلی خواهد بود.



شکل ۵ منظومه‌ی مدولاسیون M-QAM مستطیلی برای مقادیر مختلف M

کدگذاری Gray

بسته به مدولاسیونی که برای ارسال انتخاب می‌شود، هر k بیت از رشته بیت تولیدی به یک سمبل مشخص تبدیل می‌شود. به عنوان مثال شکل ۶، نگاشت طبیعی (اعداد مشکی) و گری (اعداد قرمز) را در مدولاسیون 4QAM نشان می‌دهد. همان‌طور که در شکل مشاهده می‌شود، با استفاده از کدگذاری گری، اگر تحت تأثیر نویز، یک سمبل با سمبل مجاور خود جابه‌جا شود، فقط یک بیت به اشتباه کدگذاری می‌شود؛ اما در صورت استفاده از کدگذاری طبیعی امکان دارد که هر دو بیت به اشتباه کدگذاری شود. از این رو معمولاً از کدگذاری گری استفاده می‌شود.



شکل ۶ نگاشت طبیعی (عددهای مشکی) و نگاشت گری (عددهای قرمز)

در کد گری، هر دو دنباله‌ی متوالی فقط در یک بیت با یکدیگر اختلاف دارند. برای تولید دنباله‌ی n بیتی این کد به روش زیر عمل می‌کنیم.

$n = 1$	$n = 2$	$n = 3$	$n = 4$	...
$A_1 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}_{2 \times 1}$	$A_2 = \begin{bmatrix} 0_{2 \times 1} & A_1 \\ 1_{2 \times 1} & A_1^B \end{bmatrix}_{4 \times 2}$	$A_3 = \begin{bmatrix} 0_{4 \times 1} & A_2 \\ 1_{4 \times 1} & A_2^B \end{bmatrix}_{8 \times 3}$	$A_4 = \begin{bmatrix} 0_{8 \times 1} & A_3 \\ 1_{8 \times 1} & A_3^B \end{bmatrix}_{16 \times 4}$	...

در روند ذکر شده، ماتریس A_i^B از معکوس کردن هر یک از ستون‌های A_i به دست می‌آید. به عبارتی عملگر ماتریسی B به صورتی که در ادامه آمده است، عمل می‌نماید.

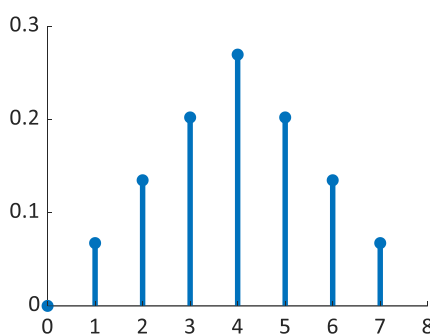
$$A_i = \begin{bmatrix} a_{1,1} & a_{1,2} & \dots & a_{1,i} \\ a_{2,1} & a_{2,2} & \dots & a_{2,i} \\ \vdots & \vdots & \ddots & \vdots \\ a_{2^i,1} & a_{2^i,2} & \dots & a_{2^i,i} \end{bmatrix} \Rightarrow A_i^B = \begin{bmatrix} a_{2^i,1} & a_{2^i,2} & \dots & a_{2^i,i} \\ a_{2^{i-1},1} & a_{2^{i-1},2} & \dots & a_{2^{i-1},i} \\ \vdots & \vdots & \ddots & \vdots \\ a_{1,1} & a_{1,2} & \dots & a_{1,i} \end{bmatrix}$$

اعمال شکل دهی پالس

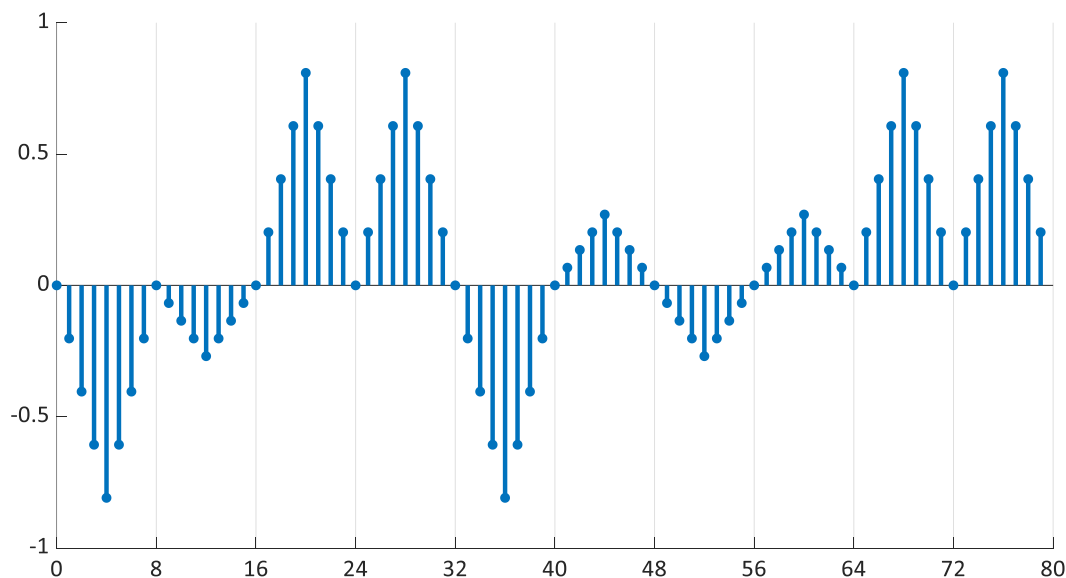
پس از انتخاب سمبل ها لازم است که از شکل دهی پالس برای ارسال سمبل ها استفاده کنیم. سیگنال شکل دهنده ی پالس را با $p(t)$ نشان می دهیم. شکل ۷، $p(t)$ مثلثی را نشان می دهد.

فرض کنید که می خواهیم یک دنباله ی ۱۰ تایی از سمبل های 4PAM را توسط $p(t)$ نشان داده در شکل ۷ ارسال کنیم. در صورتی که دنباله ی سمبل ها به صورت $s = -3, -1, 3, 3, -3, 1, -1, 1, 3, 3$ باشد، سیگنال ارسالی به صورت ۸ خواهد بود.

برای تشکیل سیگنال ارسالی، دو روش ضرب کرونیگر^۶ و استفاده از کانولوشن وجود دارد که در ادامه به توضیحاتی در مورد آن ها می پردازیم.



شکل ۷ شکل دهنده ی پالس مثلثی



شکل ۸ سیگنال ارسالی با استفاده از شکل دهنده ی پالس مثلثی

تولید سیگنال ارسالی با استفاده از ضرب کرونیگر

ضرب کرونیگر دو ماتریس A و B را که با $A \oplus B$ نشان می دهیم، به صورت زیر تعریف می شود:

^۶ Kronecker Product

$$\mathbf{A}_{m \times n} \oplus \mathbf{B}_{p \times q} = \begin{bmatrix} A_{11}\mathbf{B} & \dots & A_{1n}\mathbf{B} \\ \vdots & \ddots & \vdots \\ A_{m1}\mathbf{B} & \dots & A_{mn}\mathbf{B} \end{bmatrix}$$

در صورتی که $\mathbf{A}$ و $\mathbf{B}$ هر دو بردارهای ستونی باشند، حاصل این عملگر به صورت زیر خواهد بود:

$$\mathbf{A}_{n \times 1} \oplus \mathbf{B}_{q \times 1} = \begin{bmatrix} A_1\mathbf{B} \\ A_2\mathbf{B} \\ \vdots \\ A_n\mathbf{B} \end{bmatrix}$$

بنابراین در اینجا می‌توان سیگنال ارسالی شکل ۸ را با استفاده از ضرب کرونیگر تولید کرد.

تولید سیگنال ارسالی با استفاده از کانولوشن

دنباله‌ی سمبل‌ها را می‌توان به صورت $s = \sum_{i=0} s_i \delta[n - i]$ نشان داد. s را می‌توان به صورت زیر Zero Pad نمود:

$$s' = \sum_{i=0} s_i \delta[n - iL], \quad L = \text{length}(p)$$

حاصل کانولوشن سیگنال s' و p به صورت زیر خواهد بود که همان نتیجه‌ی مطلوب را به دست می‌دهد:

$$s' * p = \left(\sum_{i=0} s_i \delta[n - iL] \right) * p = \sum_{i=0} s_i p[n - iL]$$

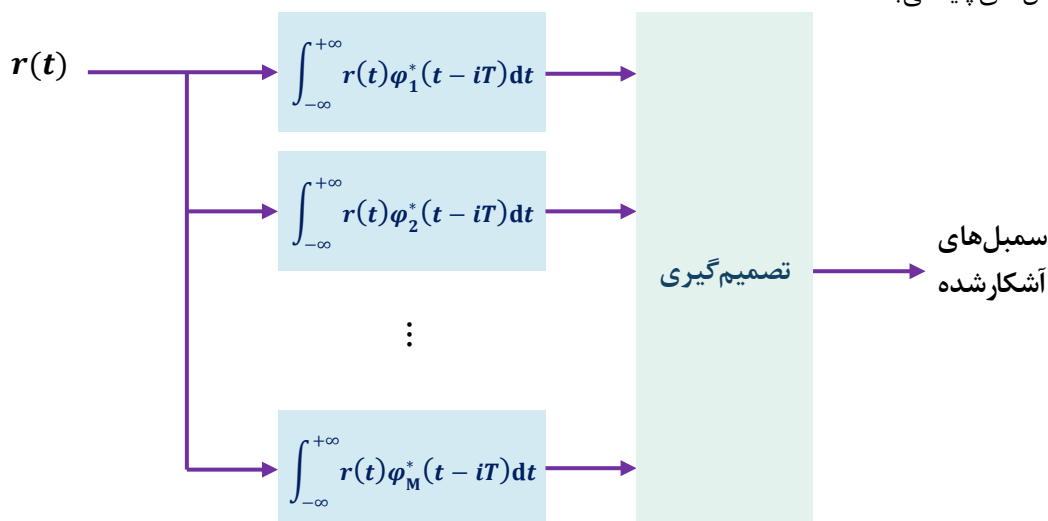
نکته: در صورتی که پالس‌های ارسالی با یکدیگر تداخل داشته باشند، روش ضرب کرونیگر سیگنال صحیح را تولید نمی‌کند.

تشخیص تصویر سیگنال دریافتی در راستای سیگنال‌های پایه

از آنجا که سمبل‌های ارسالی توسط سیگنال‌های پایه ارسال شده‌اند، می‌بایست در گیرنده، قدرت سیگنال در راستای هر یک از پایه‌ها به دست آید تا سیگنال دریافتی را آشکار کنیم. این کار با استفاده از Correlator و یا فیلتر منطبق^۷ انجام می‌شود.

Correlator

شکل ۹ شمای کلی یک گیرنده را با استفاده از Correlator (بلوک‌های آبی رنگ) نشان می‌دهد. خروجی هر یک از بلوک‌های خاکستری، قدرت سیگنال را در راستای یکی از سیگنال‌های پایه نشان می‌دهد. این مقادیر پس از ورود به بلوک تصمیم‌گیری، با منظومه‌ی سیگنالی مدولاسیون مقایسه می‌شوند و نزدیک‌ترین سمبل به آن به عنوان سمبل ارسالی انتخاب می‌شود. در بلوک‌های مشخص شده، T طول زمانی سیگنال‌های پایه می‌باشند.



شکل ۹ استفاده از Correlator برای اندازه‌گیری قدرت سیگنال در راستای هر سیگنال پایه

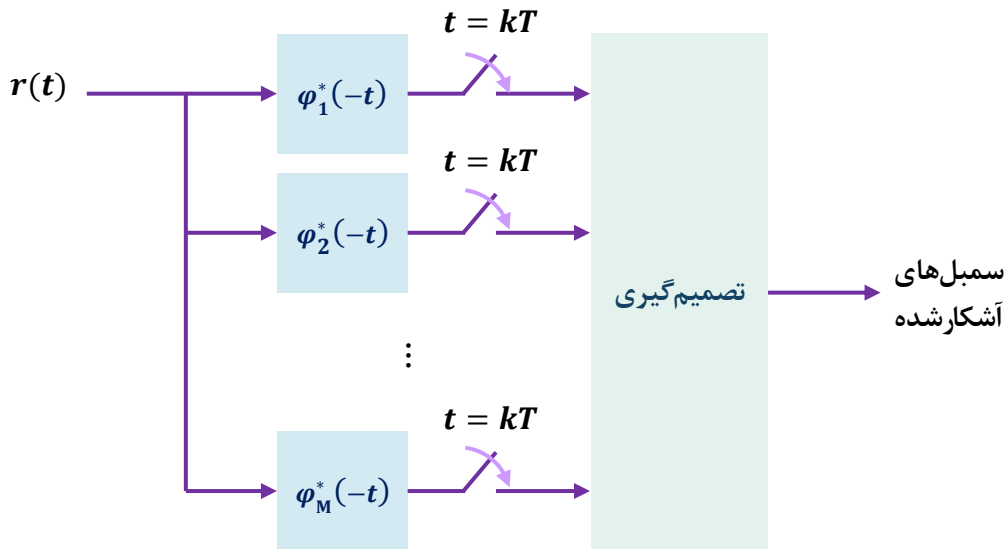
^۷ Matched Filter

فیلتر منطبق

اگر بنا باشد به جای استفاده از انتگرال گیر، از فیلتری استفاده شود که خروجی آن شبیه Correlator است، به صورت زیر عمل می شود.

$$\int_{-\infty}^{\infty} r(t) \cdot \phi_m^*(t - iT) dt = \int_{-\infty}^{\infty} r(\tau) \cdot \phi_m^*(\tau - iT) d\tau = r(t) * \phi_m^*(-t)|_{t=iT}$$

با توجه به رابطه ی فوق می توان به جای استفاده از انتگرال گیر، فیلتر $\phi_m^*(-t)$ را بر سر راه سیگنال دریافتی قرار داد و در نقاط iT از آن نمونه گیری کرد. شکل ۱۰، ساختار کلی یک گیرنده را با استفاده از Matched Filter نشان می دهد.



شکل ۱۰ استفاده از فیلتر منطبق برای اندازه گیری قدرت سیگنال در راستای هر سیگنال پایه

گیرنده ی حداقل فاصله

به گیرنده ای که سمبل دریافتی را با منظومه ی سیگنالی یک مدولاسیون مقایسه و نزدیکترین سمبل به سیگنال دریافتی را از میان سمبل های موجود در منظومه ی سیگنالی انتخاب کند، گیرنده ی حداقل فاصله گفته می شود.

شرح آزمایش

آزمایش ۱-۳: پیاده‌سازی فرستنده مدولاسیون PAM

در این آزمایش بنا داریم تعدادی بسته‌ی داده را برای ارسال از طریق مدولاسیون 4PAM آماده نماییم. از آن جا که در آزمایش‌های آینده، بناسست این فرستنده با استفاده از رادیو نرم‌افزار ADALM-PLUTO پیاده‌سازی شود، بنابراین می‌بایست با استفاده از پارامترها و ملاحظات عملی، این کار انجام شود.

پارامترهایی که در این پیاده‌سازی اهمیت دارند شامل نرخ ارسال بیت (bitRate)، ضریب افزایش نرخ نمونه‌برداری سمبل‌ها یا تعداد نمونه‌های هر سمبل (nSymbolSamples)، تعداد سمبل‌های ارسالی در هر بسته‌ی داده (nPacketSymbols) و مدت زمان ارسال داده ($\text{dataTransferDuration}$) است. در این آزمایش مقدار پارامترهای مورد نیاز به صورت جدول ۱ می‌باشد.

جدول ۱ پارامترهای آزمایش ۱-۳

پارامتر	nSymbolSamples	nPacketSymbols
مقدار	۸	۱۰

۱. تولید بیت: با استفاده از تابع bitGenerator تعدادی بیت ۰ و ۱ را متناسب با طول بسته‌ی داده و با در نظر گرفتن پارامترهای مدولاسیون 4PAM تولید نموده و آن را درون ماتریس txBit ذخیره نمایید. ماتریس txBit یک ماتریس با اندازه‌ی $\text{nPacketSymbols} \times k$ (۱۰×۲) می‌باشد. سطر اول این تابع باید به صورت زیر باشد:

```
function txBit = bitGenerator(nPacketSymbols, k)
```

۲. نگاشت بیت به سمبل:

گام ۱. تولید دنباله‌ی کد گری به صورت باینری: با استفاده از روش بیان شده برای تولید دنباله‌ی کدگذاری گری در قسمت تئوری، تابعی بنویسید که آرگومان ورودی آن تعداد بیت‌های دنباله باشد و خروجی آن کد گری مربوط به این دنباله باشد. سطر اول این تابع باید به صورت زیر باشد.

```
function [grayMatrix] = grayMatrixGenerator(k)
```

گام ۲. کدگذاری گری بیت‌های تولیدی: با استفاده از تابع $\text{grayMatrixGenerator}$ ، ماتریس مربوط به کدگذاری گری باینری مدولاسیون 4PAM را تولید کرده و در ماتریس grayMatrix ذخیره کنید. بردار جدیدی متناظر با ماتریس txBit و با نام txSymbolIndex تولید نمایید و نگاشتی یک‌به‌یک بین سطرهای ماتریس txBit و ماتریس grayMatrix برقرار کنید. به عبارتی سطر i ام بردار txSymbolIndex ، شماره‌ی سطری از grayMatrix را نشان می‌دهد که برابر با سطر i ام ماتریس txBit باشد.

راهنمایی: می‌دانیم که در مدولاسیون k تایی به $M = 2^k$ سمبل نیاز است. از طرفی برای هر سمبل نیز یک کد گری وجود دارد. بنابراین می‌توانیم کد گری اول (سطر اول grayMatrix) را به سمبل شماره ۰ و... کد گری آخر (سطر آخر grayMatrix) را به سمبل شماره $M - 1$ اختصاص دهیم. برای این منظور symbolIndex را به صورت زیر تعریف کنید (هم بیانگر شماره سمبل‌ها و هم بیانگر شماره سطر grayMatrix منتهای یک است):

```
symbolIndex = [0:M-1].';
```

سپس txBit و grayMatrix را به دسیمال تغییر داده و به ترتیب در txBitDec و grayMatrixDec ذخیره نمایید. در این صورت مانند جدول زیر، هر درایه txSymbolIndex ، برابر symbolIndex ای است که grayMatrixDec متناظر با آن برابر txBitDec متناظر با txSymbolIndex باشد.

grayMatrix	grayMatrixDec	symbolIndex
0 0	0	0
0 1	1	1
1 1	3	2
1 0	2	3

txBit	txBitDec	txSymbolIndex
0 0	0	0
1 1	3	2
0 0	0	0
0 1	1	1
1 0	2	3

گام ۳. تولید سمبل‌های ارسالی: با استفاده از تابع **constellation**، تمامی سمبل‌های یک مدولاسیون 4PAM را تولید کنید. این تابع باید به صورت زیر باشد (در آزمایش‌های بعدی این تابع را برای مدولاسیون‌های دیگر گسترش خواهیم داد):

```
function [cons, symbolAverageEnergy] = constellation(M, modulation)

modulation = lower(modulation);

switch modulation
    case 'pam'
        ...
end
end
```

که در آن **cons** بیانگر مقدار سمبل در مدولاسیون مربوطه (توجه کنید که **symbolIndex** بیانگر شماره سمبل بود) و **symbolAverageEnergy** بیانگر انرژی متوسط **constellation** بوده و باید برابر یک باشد. اکنون با استفاده از تابع **constellation** سمبل‌های متناظر با هر سطر بردار **txSymbolIndex** را تولید کرده و این سمبل‌ها را در بردار **symbolArray** ذخیره نمایید (یعنی **txSymbolIndex** بیانگر شماره اندیس سمبل‌های ارسالی و **symbolArray** بیانگر مقدار سمبل‌های ارسالی است).

۳. شکل‌دهی پالس ارسالی:

گام ۱. ساخت پالس: تابعی بنویسید که نام پالس (**pulseName**)، نرخ نمونه‌برداری (**fs**) و تعداد نمونه‌های هر سمبل (**nSymbolSamples**) را دریافت کند و پالس مربوطه را به همراه انرژی آن (مقدار انرژی باید ۱ باشد) تولید کند. این تابع باید به صورت زیر باشد (نام پالس در این آزمایش پالس مثلثی است و در آزمایش‌های بعدی این تابع را برای شکل پالس‌های دیگر گسترش خواهیم داد):

```
function [p, pulseAverageEnergy] = pulseShape (pulseName, fs, nSymbolSamples,
varargin)

pulseName = lower(pulseName);

switch pulseName
    case 'triangular'
        ...
end
end
```

گام ۲. ساخت سیگنال ارسالی: همان‌طور که در بخش تئوری بیان گردید، سیگنال ارسالی را می‌توان با دو روش ضرب کرونیگر و کانولوشن تولید و پالس ارسالی را شکل‌دهی نمود. تابعی بنویسید که بردار شماره‌ی سمبل‌های تولیدشده (**txSymbolIndex**)، مدولاسیون (**modulation**)، مرتبه‌ی مدولاسیون (**M**)، نرخ نمونه‌برداری (**fs**)، تعداد نمونه‌های هر سمبل (**nSymbolSamples**)، نام پالس (**pulseName**) و روش تولید شکل‌دهی پالس (**pulseShapingMode**) را دریافت کند و نمونه‌های سیگنال ارسالی متناظر با یک بسته و منظومه‌ی سیگنالی را تولید کند. این تابع باید به صورت زیر باشد:

```
function [txSamples, cons] = pulseModulation(txSymbolIndex, modulation, M, fs,
nSymbolSamples, pulseName, pulseShapingMode, varargin)

modulation = lower(modulation);
```

```

pulseShapingMode = lower(pulseShapingMode);
[cons, ~] = constellation(M, modulation);
symbolArray = ...
[p, ~] = pulseShape(pulseName, fs, nSymbolSamples);
switch pulseShapingMode
    case 'kron'
        ...

    case 'conv'
        ...
end
end
end

```

توجه داشته باشید که گام ۳ سوال قبل و گام ۱ همین سوال را مجدد در قالب یک تابع منسجم انجام داده ایم. در این جا **pulseShapingMode** روش شکل دهی پالس می باشد که یکی از دو حالت 'kron' و 'conv' را اختیار می کند. با استفاده از این تابع نمونه های ارسالی مربوط به یک بسته ی داده را آماده ی ارسال کرده و حاصل را در بردار **txSamples** ذخیره نمایید.

(راهنمایی ۱: ضرب کرونیگر در برنامه ی MATLAB با استفاده از دستور **kron** انجام می شود که شیوه ی استفاده از آن به صورت **txSamples = kron(symbolArray, p)** می باشد.)

(راهنمایی ۲: برای انجام عمل Zero Padding از تابع **upsample** نرم افزار MATLAB استفاده کنید. دقت نمایید که تابع شما نباید بعد از آخرین سمبل صفر اضافه نماید. حال سیگنال ارسالی را توسط عملگر کانولوشن و به صورت **txSamples = conv(symbolArrayZeroPad, p)** تولید نمایید.)

آزمایش ۲-۳: پیاده سازی گیرنده ی مدولاسیون PAM

در ادامه ی می خواهیم گیرنده ی یک مدولاسیون PAM را پیاده سازی نماییم. برای این منظور فرض کنید که نمونه های دریافتی (**rxSamples**) دقیقاً برابر نمونه های ارسالی باشند (حالت ایده آل). یعنی قرار دهید:

```
rxSamples = txSamples;
```

۱. تشخیص قدرت سیگنال دریافتی در راستای سیگنال های پایه: تابعی بنویسید که دو بردار **rxSamples** و **p** را دریافت کند و قدرت سیگنال را در راستای **p** محاسبه کند. این تابع باید بتواند قدرت سیگنال را در راستای **p**، با استفاده از هر دو روش **correlator** و فیلتر منطبق محاسبه کند. سطر اول این تابع باید به صورت زیر باشد:

```
function [rxSymbols] = symbolDetection(rxSamples, p, nSymbolSamples, rxMode)
```

در این تابع **rxMode** می تواند یکی از دو حالت 'correlator' و یا 'matchedFilter' را اختیار کند.

۲. گیرنده ی حداقل فاصله: تابعی بنویسید که منظومه ی سیگنالی (**cons**) و بردار سمبل های دریافتی (**rxSymbols**) را به عنوان ورودی بگیرد و در خروجی، برداری به طول **rxSymbols** تولید کند. المان *i*ام بردار سمبل های آشکار شده ی **detectedSymbols**، نزدیکترین مقدار **cons** به المان *i*ام بردار **rxSymbols** خواهد بود. سطر اول این تابع به صورت زیر می باشد:

```
function [detectedSymbols] = minDistanceDetector(rxSymbols, cons);
```

راهنمایی: در نرم افزار MATLAB می توان دو بردار سطری و ستونی را با یکدیگر جمع نمود! به عنوان مثال:

$$A = [a_1, a_2], \quad B = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} \rightarrow A + B = \begin{bmatrix} a_1 + b_1 & a_2 + b_1 \\ a_1 + b_2 & a_2 + b_2 \\ a_1 + b_3 & a_2 + b_3 \end{bmatrix}$$

۳. دمدولاسیون یا آشکارسازی پالس: در این جا می خواهیم تابعی بنویسیم که دو عمل قبل را به صورت منسجم در قالب یک تابع انجام دهد. ورودی های این تابع نمونه های سیگنال دریافتی (**rxSamples**)، نام مدولاسیون (**modulation**)، مرتبه ی مدولاسیون (**M**)، نرخ نمونه برداری (**fs**)، تعداد نمونه های هر سمبل (**nSymbolSamples**)، نام پالس (**pulseName**)

و روش تشخیص قدرت سیگنال دریافتی در راستای سیگنال‌های پایه (**rxMode**) بوده و خروجی‌های آن سمبل‌های دریافتی (**rxSymbols**) و اندیس سمبل‌های آشکارشده (**detectedSymbolsIndex**) است. این تابع باید به صورت زیر باشد:

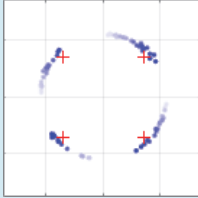
```
function [detectedSymbolsIndex, rxSymbols] = pulseDemodulation(rxSamples,
modulation, M, fs, nSymbolSamples, pulseName, rxMode, varargin)
    modulation = lower(modulation);

[cons, ~] = constellation(M, modulation);
[p, ~] = pulseShape(pulseName, fs, nSymbolSamples);
rxSymbols = symbolDetection(rxSamples, p, nSymbolSamples, rxMode);
detectedSymbols = minDistanceDetector(rxSymbols, cons);
...
end
```

۴. تبدیل از کدگذاری گری به کدگذاری باینری: به کمک بردار **detectedSymbolsIndex** و ماتریس **grayMatrix** اندیس سمبل‌های آشکارشده‌ی با کدگذاری باینری را به دست آوردید و درون متغیر **detectedBits** قرار دهید. برای این منظور متناظر با محتوای هر سطر **detectedSymbolsIndex** سطری متناظر با آن محتوا از **grayMatrix** انتخاب می‌شود.



- [1] J. G. Proakis and M. Salehi, *Digital Communications*. Boston: McGraw-Hill, 2008.
- [2] M. Rice, *Digital Communications: A Discrete-Time Approach*. Upper Saddle River, NJ: Pearson Education, 2009.
- [3] R. W. Stewart, K. Barlee, L. Crockett, and D. Atkinson, *Software Defined Radio using MATLAB & Simulink and the RTL-SDR*. 2015.



آزمایش چهارم

کاوش در مدولاسیون دیجیتال خطی

اهداف آزمایش

سامانه‌های مخابرات دیجیتال بی‌سیم اطراف ما را فرا گرفته است و ارتباطات صدا و داده در سامانه‌های مخابرات سلولی، WiFi، Bluetooth و سایر استانداردها را فراهم می‌کند. در طی چندین آزمایش آینده بلوک‌های پایه‌ی مخابرات دیجیتال شامل الگوهای مدولاسیون، دریافت سمبل و بیت، عملکرد در حضور نویز، کدگذاری و کدگشایی تقاضایی، تبدیل فرکانس به بالا و پایین دیجیتال و ... ارائه می‌شود. در این آزمایش در ادامه‌ی آزمایش قبل با الگوهای مدولاسیون دیجیتال خطی و شکل‌دهی پالس آشنا می‌شویم و به کاوش بر روی پارامترهای مختلف این مدولاسیون‌ها می‌پردازیم. هم‌چنین تأثیراتی که کانال می‌تواند بر روی فرآیند آشکارسازی بگذارد نیز مورد بررسی قرار می‌گیرد.

در انتهای این آزمایش دانشجو می‌بایست:

- منظومه‌های مدولاسیون‌های خطی M-PAM، M-PSK و M-QAM را بشناسد.
- بتواند رشته‌های بیت را به سمبل‌های مدولاسیون‌های مختلف در حالت سمبل‌های مختلط و سمبل‌های I/Q تبدیل نماید.
- بر روی کدگذاری Gray برای مدولاسیون‌های مختلف مسلط باشد.
- با علت استفاده از شکل‌دهی پالس در سمبل‌های ارسالی آشنا شود.
- با اثراتی که شکل‌دهی پالس بر روی پهنای باند طیف ارسالی می‌گذارد آشنا شود.
- بتواند سیگنال را در یک کانال باندپایه ارسال نماید و اثر نویز را در تبدیل سمبل‌ها به رشته‌های بیت بشناسد.
- بتواند به کانال باندپایه، تأخیر و اختلاف فاز اضافه نماید و با اثر این عوامل بر روی منظومه‌ی سیگنالی و خطای آشکارسازی آشنا شود.
- با طراحی Correlator و فیلتر منطبق آشنا شود.
- بتواند سمبل‌های دریافتی مدولاسیون‌های مختلف را به رشته‌های بیت تبدیل نماید.
- بتواند گیرنده‌ی حداقل فاصله را طراحی نماید.

ابزارهای مورد نیاز

- رایانه
- نرم‌افزار MATLAB R2020b
- رادیو نرم‌افزار ADALM-PLUTO
- کابل کوکسیال و آنتن



تئوری

تعیین نسبت انرژی متوسط بیت به چگالی نویز

به منظور بررسی رفتار و عملکرد مدولاسیون‌های دیجیتال خطی در عبور از یک کانال با نویز سفید جمع‌شونده، باید پارامتر نسبت انرژی متوسط بیت به چگالی نویز (E_b/N_0) به درستی تعریف شود. انرژی متوسط بیت را می‌توان بر اساس تعداد بیت‌های معرف یک سمبل و انرژی متوسط سمبل یک مدولاسیون دیجیتال تعیین نمود. با داشتن نسبت $\frac{E_b}{N_0}$ می‌توان N_0 را محاسبه نمود. البته ذکر این نکته حائز اهمیت است که در سامانه‌های عملی، ابتدا مقدار N_0 که به پارامترهای گیرنده ارتباط دارد محاسبه شده و سپس توان فرستنده برای رسیدن به انرژی متوسط بیت مورد نظر تعیین می‌گردد. از آن‌جا که برای محاسبه‌ی عملکرد یک مدولاسیون در محیط AWGN تنها نسبت E_b/N_0 اهمیت دارد، در شبیه‌سازی استفاده از هر یک از دو رویکرد فوق چندان تفاوتی ندارد.



شرح آزمایش

آزمایش ۱-۴: مقداردهی‌های اولیه

در این آزمایش بنا داریم تعدادی بسته‌ی داده را برای ارسال از طریق مدولاسیون‌های خطی آماده نماییم. از آن‌جا که در این آزمایش بناسست این فرستنده با استفاده از رادیو نرم‌افزار ADALM-PLUTO پیاده‌سازی شود، می‌بایست با استفاده از پارامترها و ملاحظات عملی، این کار انجام شود.

پارامترهایی که در این پیاده‌سازی اهمیت دارد شامل نرخ نمونه‌برداری (fs)، ضریب افزایش نرخ نمونه‌برداری سمبل‌ها یا تعداد نمونه‌های هر سمبل ($nSymbolSamples$)، تعداد سمبل‌های ارسالی در هر بسته‌ی داده ($nPacketSymbols$) و مدت زمان ارسال داده ($dataTransferDuration$) می‌باشد. در این آزمایش مقدار پارامترهای مورد نیاز به صورت جدول ۱ می‌باشد.

جدول ۱ پارامترهای آزمایش ۴

پارامتر	fs	$nSymbolSamples$	$nPacketSymbols$	$dataTransferDuration$
مقدار	10MHz	۸	شبیه‌سازی: ۱۰۰۰۰۰ سخت‌افزار: ۱۰۰۰	۱۰۰ ثانیه

دقت نظر داشته باشید که در هر آزمایش تنها می‌خواهیم برنامه‌هایی که در آزمایش قبل نوشته شده است را تکمیل نماییم. هم‌چنین ابتدای برنامه‌نویسی $nPacketSymbols$ را مقدار کوچکی مانند ۱۰ قرار دهید و پس از اطمینان از برنامه مقدار آن را افزایش دهید.

- تولید M-File مربوط به مقدار دهی اولیه: در آزمایش‌ها می‌خواهیم، مدولاسیون‌های مختلف، عوامل غیریافته‌آل کانال، استفاده یا عدم استفاده از سخت‌افزار، شکل پالس‌های مختلف و ... را مورد بررسی قرار دهیم. از این رو بهتر است برنامه به صورت کلی نوشته شود تا با تغییر چند پارامتر در فایل مقداردهی اولیه، تغییرات مد نظر در کل برنامه اعمال شود. در ادامه قالب کلی این پارامترها آورده شده است. فایل `dclInitialize.m` را ایجاد نموده و این موارد را در آن وارد نمایید. سپس باید این فایل را از برنامه‌ی اصلی خود فراخوانی نمایید.

% Changeable parameters:

```
nSymbolSamples = 8;
nPacketSymbols = 1e6;
```

```
% Number of samples in each symbol.
% Number of symbols in each packet.
```

% Modulation specification:

```
modulation = 'psk';
k = 2;
pulseName = 'triangular';
```

```
% Modulation name ('psk', 'pam', 'qam', 'fsk')
% Bit per symbol.
% Name of pulse shaping function.
```



```

pulseShapingMode = 'conv';           % Choose between convolution and Kronecker
product.                             % 
detectionMode = 'coherent';          % Modulation detection option ('coherent',
'noncoherent').                      % 
beta = 0.99;                         % Parameter for RC, RRC and Gaussian Pulse
Shape                                % 
nSymbolSpan = 1;                     % Truncation Length for RC, RRC and Gaussian
Pulse Shape (Multiple of Symbol Time)

% Flags control (0->Inactive , 1->Active):
isGray = 1;                          % Flag for having Gray code.
isHeader = 0;                        % Flag for having packets with header.
isHardware = 0;                      % Flag for using ADALM-PLUTO.

% SNR Bound for BER Plots:
snrMin = 0;                          % Minimum SNR (dB).
snrMax = 10;                         % Maximum SNR (dB).

% Channel parameters:
channelDelayInSample = round(0* nSymbolSamples); % Channel delay in sample.
channelPhaseOffset = 0 * pi/180;      % Channel phase offset.
channelFrequencyOffset = 0;           % Channel frequency offset.

% Receiver parameters:
rxMode = 'matchedFilter';             % Receiver type ('matchedFilter' or 'correlator').
channelPhaseMode = 0;                 % 0: We dont have random phase for
txSamplesDelayed and 1: we have.
compensateMode = 2;                   % 0: No compensation, 1: Amplitude compensation,
2: Phase compensation, 3: compensation amplitude and phase.
delayMode = 0;                       % 0: We dont know the channel delay, 1: we know
(unreal).

% Plot Parameters: (0: Dont plot constellation , 1: plot constellation):
constellationPlot = 1;

% Constant Parameters:
fs = 10e6;                           % Baseband sampling rate
(65105 to 61.44e6 Hz).
ts = 1/fs;                           % Baseband sampling time.
R = 50;                              % Resistance value.
M = 2^k;                              % Modulation order.
samplesPerFrame = nPacketSymbols * nSymbolSamples; % Number of samples in
frame.
Ts = nSymbolSamples*ts;               % Symbol time.
snrStep = 1;                         % SNR step (dB).
snrDb = snrMin:snrStep:snrMax;        % SNR vector (dB).
dataTransferDuration = 100;           % Transfer duration of
data for ADALM-PLUTO.
modulation = lower(modulation);

if isHardware

    % Transmitter Parameters
    txFc = 2400e6;                    % Set Transmitter Center Frequency (AD9363:
325-3800MHz) (AD9364: 70-6000MHz)
    txGain = 0;                      % Set Transmitter Attenuation As a Negative
Gain (-89.75 to 0 dB)
    txAddress = 'usb:0';              % Set Transmitter Identification Number

    % Receiver Parameters
    rxFc = 2400e6;                    % Set Receiver Center Frequency (AD9363:
325-3800MHz) (AD9364: 70-6000MHz)
    rxGain = 20;                     % Set Receiver Gain (-4dB to 71dB)
    rxAddress = 'usb:0';              % Set Receiver Identification Number

```

```

% Initialize ADALM-PLUTO:
dev = sdrdev('Pluto'); % Create Radio Object for ADALM-PLUTO

% NOTE: comment 2 lines below after first initializing ADALM-PLUTO:
setupSession(dev)
configurePlutoRadio('AD9364'); % Configure ADALM-PLUTO Radio Firmware

% Define Transmitter Object
tx = sdrtx('Pluto', 'RadioID', txAddress); % CreateTransmitterSystem Object
tx.CenterFrequency = txFc; % Set Transmitter Center Frequency
tx.Gain = txGain; % Set Transmitter Gain
tx.BasebandSampleRate = fs; % Set Baseband Sampling Rate

% Define Receiver Object
rx = sdrrx('Pluto', 'RadioID', rxAddress); % Create Receiver System Object
rx.CenterFrequency = rxFc; % Set Receiver Center Frequency
rx.BasebandSampleRate = fs; % Set Baseband Sampling Rate
rx.SamplesPerFrame = 2*samplesPerFrame; % Samples per Each Frame (< 2^20)
rx.GainSource = 'Manual'; % AGC Settings
rx.Gain = rxGain; % Receiver Gain
rx.OutputDataType = 'double'; % Output Data Type
end

```

آزمایش ۲-۴: پیاده‌سازی فرستنده مدولاسیون

۱. تولید بیت: با استفاده از تابع `bitGenerator` تعدادی بیت ۰ و ۱ را متناسب با طول بسته‌ی داده و با در نظر گرفتن پارامترهای مدولاسیون 4PSK تولید نموده و آن را درون ماتریس `txBit` ذخیره نمایید. ماتریس `txBit` یک ماتریس با اندازه‌ی $nPacketSymbols \times k$ می‌باشد.

۲. نگاشت بیت به سمبل:

گام ۱. کدگذاری گری بیت‌های تولیدی: با استفاده از تابع `grayMatrixGenerator`، ماتریس مربوط به کدگذاری گری باینری مدولاسیون 4PSK را تولید کرده و در ماتریس `grayMatrix` ذخیره کنید. بردار جدیدی متناظر با ماتریس `txBit` و با نام `txSymbolIndex` تولید نمایید و نگاشتی یک‌به‌یک بین سطرهای ماتریس `txBit` و ماتریس `grayMatrix` برقرار کنید. به عبارتی سطر i ام بردار `txSymbolIndex`، شماره‌ی سطر i از `grayMatrix` را نشان می‌دهد که برابر با سطر i ام ماتریس `txBit` باشد.

گام ۲. کدگذاری طبیعی بیت‌های تولیدی: با استفاده از پرچم `isGray`، قابلیت‌ی را به برنامه‌ی خود اضافه نمایید که بتوان کدگذاری گری را اعمال نمود.

گام ۳. تولید سمبل‌های ارسالی: با استفاده از تابع `constellation`، تمامی سمبل‌های ارسالی مدولاسیون 4PSK را تولید کرده و سمبل‌های متناظر با هر سطر بردار `txSymbolIndex` را تولید کنید. این سمبل‌ها را در بردار `symbolArray` ذخیره نمایید.

۳. شکل‌دهی پالس ارسالی: با استفاده از تابع `pulseModulation` نمونه‌های ارسالی مربوط به یک بسته‌ی داده را آماده‌ی ارسال کرده و حاصل را در بردار `txSamples` ذخیره نمایید. تابع شکل‌دهی پالس را فعلاً مثلی در نظر بگیرید.

آزمایش ۳-۴: مدل‌سازی کانال

۱. افزودن تأخیر در کانال: برای شبیه‌سازی تأخیر در کانال به اندازه‌ی پارامتر `channelDelayInSample` به ابتدای بردار `txSamples` صفر اضافه نمایید. حاصل را درون بردار `txSamplesDelayed` قرار دهید. دقت نمایید که طول بردار افزایش می‌باید و می‌بایست در ادامه‌ی برنامه اثر این افزایش لحاظ شود. در این جا فعلاً مقدار تأخیر را برابر با صفر قرار دهید.

۲. اعمال اختلاف فاز در کانال: برای اعمال اختلاف فاز در کانال، بردار `txSamplesDelayed` را در `exp(1i*channelPhaseOffset)` ضرب کرده و حاصل را در بردار `rxSamplesWithoutNoise` قرار دهید. در این جا نیز آفست فاز را برابر با صفر قرار دهید.

۳. شبیه سازی کانال با نویز سفید گاوسی

گام ۱. تعیین واریانس نویز بر اساس نسبت سیگنال به نویز (E_b/N_0): می دانیم که بر اساس خروجی تابع `constellation`، مقدار متوسط انرژی سمبل برابر ۱ است (`symbolAverageEnergy=1`). حال با استفاده از متوسط انرژی سمبل، مقدار انرژی متوسط بیت را به دست آورده و برابر متغیر `bitAverageEnergy` قرار دهید. مقدار E_b/N_0 را برابر با 3dB در نظر بگیرید (به عبارتی پارامترهای `snrMin` و `snrMax` برابر 3 تنظیم می شوند). سپس واریانس نویز را بر اساس نسبت E_b/N_0 به دست آورید و آن را درون متغیر `noiseVariance` قرار دهید.

گام ۲. افزودن نویز به سیگنال: با استفاده از تابع `randn` یک بردار نویز مختلط با واریانس `noiseVariance` و ابعاد برابر با `txSamples` تولید نمایید و آن را `noiseSamples` نامگذاری نمایید. سپس این بردار را با بردار `rxSamplesWithoutNoise` جمع نموده و حاصل را `rxSamplesPlusNoise` بنامید.

آزمایش ۴-۴: پیاده سازی گیرنده ی مدولاسیون

در ادامه ی فایل آزمایش قبل می خواهیم گیرنده ی یک مدولاسیون خطی را پیاده سازی نماییم.

۱. دمدولاسیون یا آشکارسازی پالس: نمونه های دریافت شده به همراه نویز (`rxSamplesPlusNoise`) را که اثر تأخیر کانال در آن جبران شده را درون بردار `rxSamples` قرار دهید. برای این منظور با استفاده از متغیر `delayMode` این افزایش بردار را برای دو حالت `delayMode = 0` (حالتی که گیرنده میزان تأخیر را نمی داند) و `delayMode = 1` (حالتی که گیرنده میزان تأخیر را می داند که این حالت واقعی نیست) اصلاح کرده و حاصل را در `rxSamples` قرار دهید. سپس با استفاده از تابع `pulseDemodulation` با ورودی های نمونه های سیگنال دریافتی (`rxSamples`)، نام مدولاسیون (`modulation`)، مرتبه ی مدولاسیون (`M`)، نرخ نمونه برداری (`fs`)، تعداد نمونه های هر سمبل (`nSymbolSamples`)، نام پالس (`pulseName`) و روش تشخیص قدرت سیگنال دریافتی در راستای سیگنال های پایه (`mode`)، سمبل های دریافتی (`rxSymbols`) و اندیس سمبل های آشکار شده (`detectedSymbolsIndex`) را به دست آورید.

۲. تبدیل از کدگذاری گری به کدگذاری باینری: به کمک بردار `detectedSymbolsIndex` و ماتریس `grayMatrix` اندیس سمبل های آشکار شده ی با کدگذاری باینری را به دست آوردید و درون متغیر `detectedBits` قرار دهید. برای این منظور متناظر با محتوای هر سطر `detectedSymbolsIndex` سطری متناظر با آن محتوا از `grayMatrix` انتخاب می شود. این عمل باید برای حالتی که از کدگذاری طبیعی استفاده می شود (`isGray = 0`) نیز مدیریت شود.

۳. محاسبه ی خطای سمبل: با مقایسه ی تعداد اختلاف های بردارهای `detectedSymbolsIndex` و `symbolIndex` و محاسبه نسبت اختلاف این دو بردار به تعداد کل سمبل ها، خطای سمبل را به دست آورده و آن را در متغیر `ser` قرار دهید.

۴. تبدیل سمبل به بیت و محاسبه ی خطای بیت: با مقایسه ی تعداد اختلاف ماتریس های `detectedBits` و `txBit` و محاسبه نسبت تعداد اختلاف این دو ماتریس به تعداد کل بیت ها، خطای بیت را به دست آورده و آن را در متغیر `ber` قرار دهید.

آزمایش ۴-۵: خواسته های کلی

۱. طیف سیگنال ارسالی: طیف فرکانسی سیگنال ارسالی `txSamples` را برای مدولاسیون 4PSK رسم نمایید. در مورد پهنای باند و سطح باندهای جانبی آن توضیحاتی ارائه دهید.

۲. رسم نمودار نرخ خطای بیت: با تغییر نسبت سیگنال به نویز بین 0 تا 10dB نرخ خطای بیت را برای مدولاسیون های 16QAM، 8PSK و 4PAM را به دست آورده و در یک نمودار رسم نمایید و آن را با خروجی تابع **berawgn** نرم افزار MATLAB مقایسه نمایید. این کار را برای شکل موج مثلثی و $\text{root raised cosine}$ انجام دهید. همچنین اثر اعمال کدگذاری گری و کدگذاری باینری را نیز بر روی نمودار نرخ خطای بیت نشان دهید. با استفاده از **rxSymbols**، منظومه های سیگنالی (**constellation**) را نیز برای حالتی که کدگذاری گری اعمال شده است رسم نمایید.

۳. اثر تأخیر در کانال:

در این جا فرض نمایید مدولاسیون 4PSK است و نسبت سیگنال به نویز (E_b/N_0) برابر با 10dB است.

گام ۱. اثر تأخیر در کانال بر روی نرخ خطای بیت و منظومه سیگنالی: تأخیر کانال را برابر با گرد شده ی 0.1، 0.5 و 0.8 برابر طول یک سمبل قرار دهید (یک عدد صحیح معادل تعداد نمونه های زمانی تأخیر یافته). نرخ خطای بیت را به دست آورده و با حالت بدون تأخیر مقایسه نمایید. منظومه سیگنالی حالت های فوق را نیز با یکدیگر مقایسه کرده و علت مشاهدات خود را توضیح دهید (**delayMode= 0**، **isGray = 0** و **channelPhaseOffset = 0**).

گام ۲. اضافه کردن هدر: برای یافتن نقطه ی شروع درست داده ها، تعدادی داده ی مشخص به عنوان هدر در ابتدای رشته بیت های ارسالی قرار دهید (دقت داشته باشید که طول **txBit** نباید افزایش یابد). بیت های هدر باید به صورت زیر باشد. این رشته بیت خواص خودهمبستگی خوبی دارد.

```
headerBits = repmat(reshape(de2bi(hex2dec(['1C6387FF5DA4FA325C895958DC5'])))', [1, 1], 1, k);
```

سپس نمونه های زمانی مربوط به هدر را از بردار **txSamples** جدا کرده و در متغیر **headerSamples** ذخیره نمایید. با استفاده از متغیر **isHeader**، برنامه را برای حالتی که هدر وجود ندارد مدیریت کنید.

گام ۳. تخمین زمان شروع بسته و جبران اثر تأخیر کانال: با استفاده از همبستگی بردار **rxSamplesPlusNoise** و نمونه های زمانی هدر **headerSamples**، نقطه ی شروع بسته ی داده را تخمین بزنید. با مقایسه ی نرخ خطای بیت حالت بدون تأخیر و حالتی که نقطه ی شروع داده ها تخمین زده می شود (در هر دو حالت **ber=0**)، از عملکرد برنامه ی خود اطمینان حاصل نمایید. توجه کنید که این قسمت فقط زمانی اجرا می شود که **isHeader = 1** باشد.

۴. اعمال فاز در کانال و تخمین فاز:

در این جا فرض نمایید مدولاسیون 4PSK است و نسبت سیگنال به نویز (E_b/N_0) برابر با 10dB است.

گام ۱. اثر فاز اضافی کانال بر نرخ خطای بیت: اختلاف فاز کانال را برابر ۳۰ درجه قرار دهید. ابتدا برای مدولاسیون 4PSK نرخ خطای بیت را به دست آورده و با حالت بدون اختلاف فاز مقایسه نمایید. منظومه سیگنالی هر دو حالت را نیز با یکدیگر مقایسه نمایید. علت مشاهدات خود را توضیح دهید (**delayMode = 0**، **isGray = 0**، **channelDelayInSample = 0** و **isHeader = 0**).

گام ۲. تخمین فاز اضافی کانال: اگر به ابتدای بسته ی ارسالی هدر بخش قبل اضافه شود، با استفاده از همبستگی بردار **rxSamplesPlusNoise** و نمونه های زمانی هدر **headerSamples**، فازی را که کانال به سیگنال اضافه کرده است به دست آورید. توجه کنید که این قسمت فقط زمانی اجرا می شود که **isHeader = 1** باشد. راهنمایی: به فاز در قله ی خروجی همبستگی دقت نمایید. قله از روی مقدار مطلق همبستگی به دست می آید ولی مقدار خود همبستگی در این اندیس به دست آمده مورد استفاده قرار می گیرد. به عملکرد تابع **max** بر روی داده های مختلط توجه نمایید.

گام ۳. جبران سازی فاز کانال: تلاش کنید با اعمال ضریب مناسب اثر فاز اضافی کانال را جبران نمایید.

۵. جبران سازی: در این قسمت می خواهیم تابعی بنویسیم که عملیات انجام شده برای جبران سازی تأخیر و فاز را به صورت منسجم در قالب یک تابع انجام دهد. علاوه بر آن می خواهیم با اعمال ضریب مناسب اثر تغییر دامنه را نیز جبران کنیم. ورودی های این تابع نمونه های زمانی هدر (**headerSamples**)، نمونه های نویزی سیگنال دریافتی

(**rxSamplesPlusNoise**)، اندازه‌ی قله‌ی خودهمبستگی نمونه‌های زمانی هدر (**maxRx**) و نوع جبران‌سازی (**compensateMode**) بوده و خروجی آن نمونه‌های دریافتی پس از جبران‌سازی (**rxSignalSynch**) است. این تابع باید به صورت زیر باشد:

```
function [rxSignalSynch] = rxSyncAndCompensate(samplesPerFrame, headerSamples,
rxSamplesPlusNoise, maxRx, compensateMode)

% Time synchronization:
...
rxSignalTimeSynch = ...

% Compensation
switch compensateMode

    case 0 % No Compensation for Amplitude and Phase.
        rxSignalSynch = rxSignalTimeSynch;

    case 1 % Amplitude Compensation.
        ...

    case 2 % Phase Compensation.
        ...

    case 3 % Amplitude and phase Compensation.
        ...

end

End
```

آزمایش ۴-۶: پیاده‌سازی سخت‌افزاری

در فایل **dclInitialize.m** پس از پیکربندی رادیو نرم‌افزار یک شیء فرستنده و یک شیء گیرنده ایجاد شده است. فرکانس مرکزی شیء فرستنده و گیرنده بر روی 2400MHz، بهره‌ی فرستنده برابر با 0dB، بهره‌ی گیرنده به صورت Manual و برابر 20dB تنظیم شود. توجه کنید که به علت اطمینان از دریافت یک بسته‌ی کامل، تعداد نمونه‌های زمانی دریافتی برابر با دو برابر تعداد نمونه‌های زمانی بسته در نظر گرفته شده است. با استفاده از پرچم **isHardware** برنامه خود را برای دو حالت تئوری (برنامه‌های قبلی) و استفاده از سخت‌افزار مدیریت کنید (برای استفاده از سخت‌افزاری **isHardware = 1** می‌باشد).

۱. ارسال نمونه‌ها با استفاده از ADALM-PLUTO: بردار **txSamples** مربوط به مدولاسیون 4PSK (هدر اضافه شود) را با استفاده از دستور **transmitRepeat** به مدت زمان **dataTransferDuration** ثانیه به صورت پی‌درپی در فضا ارسال نمایید. این دستور داده‌ها را از طریق USB به رادیو نرم‌افزار ارسال می‌نماید و در حافظه‌ی سخت‌افزار ذخیره می‌نماید. فرستنده داده‌ها را از حافظه‌ی رادیو نرم‌افزار قرائت کرده و مدام ارسال می‌نماید.

۲. رسم لحظه‌ای نمودارها: برای رسم نمودارها به صورت لحظه‌ای، قبل از نوشتن حلقه تکرار برنامه خود مقادیر اولیه زیر را کامل کرده و سپس تابع **plotLiveReceiver** را که در اختیار شما قرار گرفته است فراخوانی نمایید.

```
freq = ...;
rxSymbolsReal = zeros(size(freq));
rxSymbolsImag = zeros(size(freq));
rxSamplesFftDBm = zeros(size(freq));
rxSamplesFftDBmMax = ...

plotLiveReceiver(rxSymbolsReal, rxSymbolsImag, rxSamplesFftDBm,
rxSamplesFftDBmMax, freq, modulation, M);
```

۳. حلقه‌ی تکرار برنامه‌ی خود را با استفاده از دستورهای **tic** و **toc** و بر اساس متغیر زمان توقف دریافت داده (**dataTransferDuration**) پیاده‌سازی نمایید. برای نوشتن حلقه به ترتیب زیر عمل کنید:

گام ۱. دریافت سیگنال با استفاده از ADALM-PLUTO: با استفاده از فراخوانی شیء گیرنده با وارد کردن $rx()$ ، یک فریم از ADALM-PLUTO را دریافت نموده و در متغیر `rxSamplesPlusNoise` ذخیره کنید.

گام ۲. محاسبه طیف توان و بیشینه طیف توان لحظه‌ای: با استفاده از FFT داده‌های به دست آمده از رادیونرم‌افزار (`rxSamplesPlusNoise`)، طیف توان آن‌ها را بر حسب dBm به دست آورده و در متغیر `rxSamplesFftDBm` قرار دهید. سپس بیشینه‌ی طیف توان لحظه‌ای در کل بازه‌ی زمان دریافت (Max Hold) را محاسبه کرده و در متغیر `rxSamplesFftDBmMax` قرار دهید.

گام ۳. جبران‌سازی و دمدولاسیون: با استفاده از پرچم `isHeader` عمل دمدولاسیون را برای حالتی که هدر وجود ندارد نیز مدیریت کنید. در صورت وجود هدر باید با استفاده از توابع `rxSyncAndCompensate` و `pulseDemodulation` که در جلسات قبلی نوشته‌اید ابتدا عمل جبران‌سازی و سپس دمدولاسیون را انجام دهید و در صورت عدم وجود هدر، فقط عمل دمدولاسیون را با وارد کردن `rxSamplesPlusNoise` در تابع `pulseDemodulation` انجام دهید. توجه کنید در هر دو حالت به دلیل این‌که تعداد نمونه‌های دریافتی دو برابر تعداد نمونه‌های ارسالی است، طول `detectedSymbolsIndex` با طول `txSymbolIndex` یکسان نبوده و این مساله باید مدیریت شود. پس از انجام دمدولاسیون و یافتن `rxSymbols`، قسمت حقیقی و موهومی آن را به ترتیب در متغیرهای `rxSymbolsReal` و `rxSymbolsImag` قرار دهید.

گام ۴. محاسبه خطای بیت: احتمال خطای سمبل و بیت را محاسبه کرده و در `ser` و `ber` قرار دهید. مانند بخش دوم از آزمایش ۴-۴، محاسبه احتمال خطا باید با استفاده از پرچم `isGray` برای هر دو حالت کدگذاری گری و طبیعی مدیریت شود.

گام ۵. به روز رسانی داده‌ها: برای به روز رسانی داده‌ها از دستورات `refreshdata` و `drawnow` استفاده نمایید.

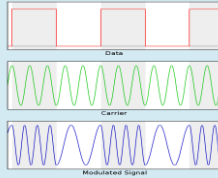
۴. اثر شکل پالس بر روی پهنای باند سیگنال ارسالی: طیف فرکانسی سیگنال `rxSamplesPlusNoise` دریافتی از ADALM-PLUTO را برای مدولاسیون 4PSK و به ازای شکل پالس‌های مثلثی و root raised cosine رسم نمایید. در مورد پهنای باند و سطح باندهای جانبی آن توضیحاتی ارائه دهید.

۵. جبران‌سازی دامنه، فاز و تأخیر و مشاهده‌ی منظومه‌ی سیگنالی: برنامه را برای حالات مختلف جبران‌سازی (`compensateMode = 0, 1, 2, 3`) اجرا کرده و منظومه‌ی سیگنالی مربوط به `rxSymbols` را رسم نمایید. در کدام حالت احتمال خطا بیشتر است؟ اگر از کدگذاری گری استفاده نکنیم وضعیت به چه صورتی خواهد بود؟ اگر از هدر استفاده نکنیم چطور؟



- [1] J. G. Proakis and M. Salehi, *Digital Communications*. Boston: McGraw-Hill, 2008.
- [2] M. Rice, *Digital Communications: A Discrete-Time Approach*. Upper Saddle River, NJ: Pearson Education, 2009.
- [3] R. W. Stewart, K. Barlee, L. Crockett, and D. Atkinson, *Software Defined Radio using MATLAB & Simulink and the RTL-SDR*. 2015.





آزمایش پنجم

مدولاسیون FSK همدوس

اهداف آزمایش

- پس از آشنایی با مدولاسیون‌های خطی و اثر یک کانال AWGN بر روی آن، در این آزمایش با مدولاسیون FSK همدوس آشنا می‌شویم و رفتار آن را دوباره در کانال AWGN مورد بررسی قرار می‌دهیم.
- در انتهای این آزمایش دانشجو می‌بایست:
- با مفهوم مدولاسیون‌های متعامد آشنا شود.
 - منظومه‌ی سیگنالی مدولاسیون FSK را بشناسد.
 - با شکل موج مدولاسیون FSK آشنا باشد.
 - بر اثر فاصله‌ی فرکانسی بین پایه‌های مدولاسیون FSK مسلط شود.
 - بتواند یک سیستم کامل مدولاسیون FSK را پیاده‌سازی کند.
 - احتمال خطای سمبل و بیت را در این مدولاسیون به دست آورد.
 - یک سامانه‌ی کامل مدولاسیون FSK همدوس را به کمک رادیو نرم‌افزار ADALM-PLUTO پیاده‌سازی نماید.

ابزارهای مورد نیاز

- رایانه 
- نرم‌افزار MATLAB R2020b 
- رادیو نرم‌افزار ADALM-PLUTO 
- آنتن 



مدولاسیون های متعامد

در مدولاسیون های متعامد، بُعد فضا برابر با تعداد نقاط منظومه ی سیگنالی می باشد. به عبارتی، در صورتی که N نشان دهنده ی بُعد فضا باشد، $N = M$ خواهد بود که $S_1(t), S_2(t), \dots, S_M(t)$ سیگنال های ارسالی می باشند. برای این مدولاسیون ها داریم:

$$\langle S_i(t), S_j(t) \rangle = \begin{cases} 0 & i \neq j \\ E_s & i = j \end{cases}$$

بدیهی است که سیگنال های پایه برای این مدولاسیون ها به صورت زیر خواهند بود:

$$\phi_i(t) = \frac{S_i(t)}{\|S_i(t)\|} = \frac{S_i(t)}{\sqrt{E_s}}$$

حالت خاصی از مدولاسیون های متعامد، مدولاسیون FSK⁸، می باشد که سیگنال های ارسالی به صورت زیر می باشند:

$$S_m(t) = \sqrt{\frac{2E_s}{T}} \cos(2\pi(f_0 + (m-1)\Delta f)t) \quad 0 < t \leq T, \quad m = 1, 2, \dots, M$$

همان گونه که مشاهده می شود، شکل دهنده ی پالس در اینجا یک مستطیل به طول T خواهد بود. معادل باند پایه برای این سیگنال ها به صورت زیر خواهد بود:

$$S_{\ell,m}(t) = \sqrt{\frac{2E_s}{T}} e^{j2\pi(m-1)\Delta f t}$$

حال می خواهیم Δf را به گونه ای تنظیم کنیم که سیگنال های ارسالی بر یکدیگر عمود باشند. داریم:

$$\begin{aligned} \langle S_{\ell,m}(t), S_{\ell,n}(t) \rangle &= \int_{-\infty}^{\infty} S_{\ell,m}(t) S_{\ell,n}^*(t) dt = \int_0^T \sqrt{\frac{2E_s}{T}} e^{j2\pi(m-1)\Delta f t} \sqrt{\frac{2E_s}{T}} e^{-j2\pi(n-1)\Delta f t} dt = \frac{2E_s}{T} \int_0^T e^{j2\pi(m-n)\Delta f t} dt \\ &= \frac{2E_s}{T} e^{j\pi(m-n)\Delta f T} \times \frac{\sin(\pi(m-n)\Delta f T)}{\pi(m-n)\Delta f} \end{aligned}$$

از طرفی داریم:

$$\langle S_m(t), S_n(t) \rangle = \frac{1}{2} \text{Re}\{\langle S_{\ell,m}(t), S_{\ell,n}(t) \rangle\}$$

بنابراین:

$$\langle S_m(t), S_n(t) \rangle = \frac{1}{2} \text{Re}\{\langle S_{\ell,m}(t), S_{\ell,n}(t) \rangle\} = E_s \frac{(\cos(\pi(m-n)\Delta f T) \times \sin(\pi(m-n)\Delta f T))}{\pi(m-n)\Delta f T} = E_s \text{sinc}(2(m-n)\Delta f T)$$

به منظور برقراری شرط تعامد لازم است که:

$$\text{sinc}(2(m-n)\Delta f T) = 0$$

بنابراین برای اینکه تعامد در باند میانی برقرار شود لازم است که شرط $\Delta f = \frac{k}{2(m-n)T}$; $k \in \mathbb{Z}$ برقرار باشد. حداقل مقدار Δf که

تعامد را برقرار می سازد برابر مقدار زیر است:

$$\Delta f_{\min} = \frac{1}{2T}$$

از آنجا که پهنای باند مصرفی با Δf متناسب است، معمولاً در عمل سعی می شود که از $\Delta f_{\min}$ استفاده شود. لازم به ذکر است که

پایه های فضای سیگنال در باند میانی به صورت زیر می باشند:

$$\Phi_m(t) = \sqrt{\frac{2}{T}} \times \cos(2\pi(f_0 + (m-1)\Delta f)t), \quad 0 < t \leq T$$

به منظور دمدولاسیون و آشکارسازی در باند پایه، فرض کنید که مجموعه سیگنال های ارسالی به صورت

$S_m(t) = g_m(t) \cos(2\pi f_0 t)$ باشند. اگر معادل باند پایه سیگنال $S_m(t)$ را با $S_{\ell,m}(t)$ نشان دهیم، خواهیم داشت:

$$S_m(t) = \Re\{S_{\ell,m}(t) e^{j2\pi f_0 t}\}$$

سیگنال دریافتی معادل باند پایه برای این سیستم به صورت زیر است:

⁸ Frequency Shift Keying



$$r_\ell(t) = S_{\ell,m}(t) + n_\ell(t)$$

می توان اثبات کرد که تصمیم گیری بهینه برای اینکه احتمال خطا مینیمم باشد به صورت زیر است:

$$\hat{m} = \underset{m=1,\dots,M}{\operatorname{argmax}} \left\{ \operatorname{Re}(\langle \underline{r}_\ell, \underline{S}_{\ell,m} \rangle) + N_0 \ln \pi_m - \frac{1}{2} E_{\ell,m} \right\}$$

در رابطه‌ی فوق $\underline{r}_\ell$ معادل برداری سیگنال دریافتی در باند پایه، $\underline{S}_{\ell,m}$ معادل برداری سیگنال ارسالی m ام در باند پایه و π_m احتمال

ارسال سمبل m ام می‌باشد. عملگر $\langle \cdot \rangle$ معادل ضرب داخلی دو بردار است که به صورت زیر تعریف می‌شود:

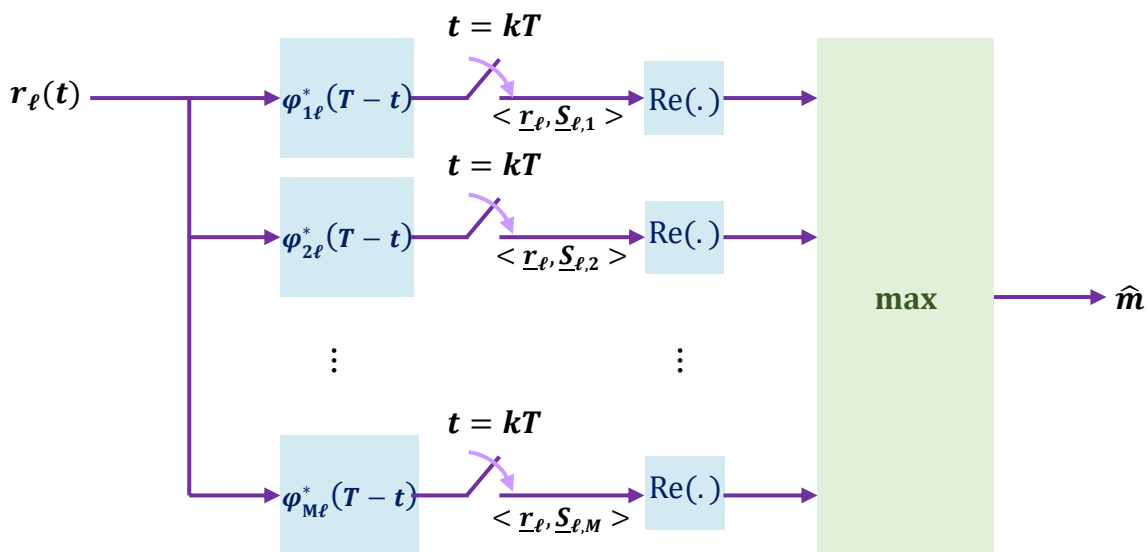
$$\underline{X} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix}, \underline{Y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} \rightarrow \langle \underline{X}, \underline{Y} \rangle = \underline{Y}^H \underline{X}$$

بدین ترتیب اگر احتمال ارسال همه سمبل‌ها برابر و انرژی سیگنال ارسالی متناظر با هر کدام از آن‌ها یکسان باشد، عبارت فوق

به صورت زیر ساده خواهد شد:

$$\hat{m} = \underset{m=1,\dots,M}{\operatorname{argmax}} \{ \operatorname{Re}(\langle \underline{r}_\ell, \underline{S}_{\ell,m} \rangle) \}$$

گیرنده متناظر با این تصمیم‌گیری به صورت زیر خواهد بود:



شکل ۱ شمای گیرنده‌ی همدوس

شرح آزمایش

آزمایش ۱-۵: مقداردهی‌های اولیه

در این آزمایش بنا داریم تعدادی بسته‌ی داده را برای ارسال از طریق مدولاسیون FSK آماده نماییم. از آن‌جا که در این آزمایش بناس‌ت این فرستنده با استفاده از رادیو نرم‌افزار ADALM-PLUTO پیاده‌سازی شود، می‌بایست با استفاده از پارامترها و ملاحظات عملی، این کار انجام شود.

پارامترهایی که در این پیاده‌سازی اهمیت دارد شامل نرخ نمونه‌برداری (fs)، تعداد نمونه‌های هر سمبل ($nSymbolSamples$)، تعداد سمبل‌های ارسالی در بسته‌ی داده ($nPacketSymbols$) و مدت زمان ارسال داده ($dataTransferDuration$) است. در این آزمایش مقدار پارامترهای مورد نیاز به صورت جدول ۱ می‌باشد.

جدول ۱ پارامترهای آزمایش ۵

پارامتر	fs	$nSymbolSamples$	$nPacketSymbols$	$dataTransferDuration$
مقدار	10MHz	64	شبیه‌سازی: ۱۰۰۰۰۰ سخت‌افزار: ۱۰۰۰	۱۰۰ ثانیه

دقت نظر داشته باشید که در این آزمایش نیز تنها می‌خواهیم برنامه‌هایی که در آزمایش قبل نوشته شده است را تکمیل نماییم.

۱. تولید M-File مربوط به مقدار دهی اولیه: قالب کلی پارامترهای مورد نیاز در این آزمایش، مانند آزمایش قبل است و فایل `dclInitialize.m` می‌بایست مدولاسیون `fsk` را نیز بشناسد. این فایل را باید از برنامه‌ی اصلی فراخوانی نمایید.

آزمایش ۲-۵: پیاده‌سازی فرستنده مدولاسیون FSK

۱. تولید بیت: با استفاده از تابع `bitGenerator` تعدادی بیت ۰ و ۱ را متناسب با طول بسته‌ی داده و با در نظر گرفتن پارامترهای مدولاسیون 4FSK تولید نموده و آن را درون ماتریس `txBit` ذخیره نمایید. ماتریس `txBit` یک ماتریس با اندازه‌ی $nPacketSymbols \times k$ می‌باشد.

۲. نگاشت بیت به شماره‌ی سمبل: بردار جدیدی متناظر با ماتریس `txBit` و با نام `txsymbolIndex` تولید نمایید و متناظر با هر سطر ماتریس `txBit` یک عدد صحیح معادل با آن بیت‌ها در `txsymbolIndex` قرار دهید. دقت نمایید در مدولاسیون FSK نیازی به کدگذاری گری نیست. بنابراین در مقداردهی اولیه برنامه `isGray` را برابر صفر قرار دهید.

۳. تولید نمونه‌های زمانی سمبل ارسالی: با استفاده از تابع `pulseModulation` ابتدا نمونه‌های زمانی متناظر با هر شماره‌ی سمبل را تولید و در نهایت نمونه‌های زمانی یک بسته‌ی داده را آماده‌ی ارسال کرده و حاصل را در بردار `txSamples` ذخیره نمایید. دقت نمایید که انرژی هر سمبل برابر با $E_s = 1$ باشد تا انرژی متوسط سمبل‌ها نیز برابر با $E_{s,avg} = 1$ شود. توضیحات تکمیلی: در این آزمایش باید تابع `pulseModulation` را برای استفاده در مدولاسیون FSK توسعه دهیم. این تابع را به صورت زیر توسعه دهید:

```
function [txSamples, cons] = pulseModulation(txSymbolIndex, modulation, M, ...
    fs, nSymbolSamples, pulseName, pulseShapingMode, varargin)

modulation = lower(modulation);

if modulation == 'fsk'
    ts = 1/fs; % Baseband Sampling Period
    Ts = nSymbolSamples*ts; % Pulse Duration
    t = (0:nSymbolSamples-1).'*ts;
    detectionMode = varargin{end};
```

```

detectionMode = lower(detectionMode);

switch detectionMode
    case 'coherent'
        deltaf = ...

end
...
cons = 'Nothing';

else
    کد قبلی
end
end

```

برای تکمیل تابع فوق توجه کنید که در این آزمایش نمونه‌های سیگنال در باندپایه ایجاد می‌شود. چون سیگنال در باند میانی تولید نمی‌شود نیازی به دانستن f_0 نیست. از این رو می‌بایست از فرمول معادل باندپایه‌ی سیگنال مدولاسیون FSK استفاده نمود. فرمول‌هایی که در بخش توضیحات این آزمایش نوشته شده مربوط به سیگنال زمان پیوسته است. در حالت زمان گسسته ابتدا یک بردار زمان به صورت $t = (0:nSymbolSamples-1)*ts$ ایجاد می‌کنیم که در آن $t_s = 1/f_s$ زمان نمونه‌برداری است. سپس با استفاده از این بردار که در واقع نقاط نمونه‌برداری از معادل پایین گذر است، شکل گسسته‌ی هر کدام از سمبل‌ها به دست می‌آید. باید ضریب مناسبی به جای $\sqrt{\frac{2E_s}{T}}$ گذاشته شود تا انرژی هر سمبل واحد شود. حال نمونه‌های سمبل‌های مختلف را پشت سر هم قرار می‌دهیم. در واقع اگر تعداد سمبل‌ها برابر ۱۰ باشد و **nSymbolSamples** برابر با ۶۴ باشد، بردار **txSamples** یک بردار با ۶۴۰ سطر و یک ستون خواهد بود. با استفاده از یک حلقه‌ی for می‌توان پس از تولید هر یک از سمبل‌ها، آن را در موقعیت مناسب در بردار **txSamples** قرار داد.

آزمایش ۳-۵: مدل سازی کانال

۱. افزودن تأخیر در کانال: برای شبیه‌سازی تأخیر در کانال به اندازه‌ی پارامتر **channelDelayInSample** به ابتدای بردار **txSamples** صفر اضافه نمایید. حاصل را درون بردار **txSamplesDelayed** قرار دهید. دقت نمایید که طول بردار افزایش می‌باید و می‌بایست در ادامه‌ی برنامه اثر این افزایش لحاظ شود. در این جا فعلاً مقدار تأخیر را برابر با صفر قرار دهید.
۲. اعمال اختلاف فاز در کانال: برای اعمال اختلاف فاز در کانال، بردار **txSamplesDelayed** را در $\exp(1i*channelPhaseOffset)$ ضرب کرده و حاصل را در بردار **rxSamplesWithoutNoise** قرار دهید. در این جا نیز آفست فاز را برابر با صفر قرار دهید.

۳. شبیه‌سازی کانال با نویز سفید گاوسی

۱. تعیین واریانس نویز بر اساس نسبت سیگنال به نویز (E_b/N_0) : با توجه به تولید هر سمبل با انرژی واحد، مقدار انرژی متوسط بیت را به دست آورده و برابر متغیر **bitAverageEnergy** قرار دهید. مقدار E_b/N_0 را برابر با 10dB در نظر بگیرید (به عبارتی پارامترهای **snrMin** و **snrMax** برابر 10 تنظیم می‌شوند). سپس واریانس نویز را بر اساس نسبت E_b/N_0 به دست آورید و آن را درون متغیر **noiseVariance** قرار دهید.
۲. افزودن نویز به سیگنال: با استفاده از تابع **randn** یک بردار نویز مختلط با واریانس **noiseVariance** و ابعاد برابر با **txSamples** تولید نمایید و آن را **noiseSamples** نامگذاری نمایید. سپس این بردار را با بردار غیر نویزی دریافتی **rxSamplesWithoutNoise** جمع نموده و حاصل را **rxSamplesPlusNoise** بنامید.

آزمایش ۴-۵: پیاده‌سازی گیرنده‌ی مدولاسیون FSK

در این جا می‌خواهیم گیرنده‌ی مدولاسیون FSK را پیاده‌سازی نماییم.

۱. دمدولاسیون یا آشکارسازی پالس: نمونه‌های دریافت شده به همراه نویز (**rxSamplesPlusNoise**) را که اثر تأخیر کانال در آن جبران شده را درون بردار **rxSamples** قرار دهید. برای این منظور با استفاده از متغیر **delayMode** این افزایش بردار را برای دو حالت **delayMode = 0** (حالتی که گیرنده میزان تأخیر را نمی‌داند) و **delayMode = 1** (حالتی که گیرنده میزان تأخیر را می‌داند که این حالت واقعی نیست) اصلاح کرده و حاصل را در **rxSamples** قرار دهید. سپس با استفاده از تابع **pulseDemodulation** با ورودی‌های نمونه‌های سیگنال دریافتی (**rxSamples**)، نام مدولاسیون (**modulation**)، مرتبه‌ی مدولاسیون (**M**)، نرخ نمونه‌برداری (**fs**)، تعداد نمونه‌های هر سمبل (**nSymbolSamples**)، نام پالس (**pulseName**) و روش تشخیص قدرت سیگنال دریافتی در راستای سیگنال‌های پایه (**mode**)، سمبل‌های دریافتی (**rxSymbols**) و اندیس سمبل‌های آشکار شده (**detectedSymbolsIndex**) را به دست آورید (توجه کنید که در مدولاسیون FSK پارامترهای ورودی **pulseName** و **mode** مورد استفاده قرار نمی‌گیرند). توضیحات تکمیلی: در این آزمایش باید تابع **pulseDemodulation** را برای استفاده در مدولاسیون FSK توسعه دهیم. این تابع را به صورت زیر توسعه دهید:

```
function [detectedSymbolsIndex, rxSymbols] = pulseDemodulation(rxSamples, ...
    modulation, M, fs, nSymbolSamples, pulseName, rxMode, varargin)

modulation = lower(modulation);

if modulation == 'fsk'
    ts = 1/fs; % Baseband Sampling Period
    Ts = nSymbolSamples*ts; % Pulse Duration
    t = (0:nSymbolSamples-1).*ts;
    detectionMode = varargin{end};
    detectionMode = lower(detectionMode);

    switch detectionMode
        case 'coherent'
            deltaf = ...;
    end

    ...

    switch detectionMode
        case 'coherent'
            ...
    end

else
    % کد قبلی
end
end
```

برای تکمیل تابع ابتدا یک ماتریس مانند **phiMatrix** را طوری بسازید که هر ستون آن یکی از پایه‌های متعامد فضای سیگنال باشد. سپس با استفاده از آن و تابع **symbolDetection**، ماتریس **rxSymbols** را طوری بسازید که هر ستون آن قدرت **rxSymbols** در راستای پایه فضای مربوطه در **phiMatrix** باشد. قسمت حقیقی ماتریس **rxSymbols** را با **realRxSymbols** نمایش دهید. در نهایت با استفاده از **realRxSymbols** و نحوه آشکارسازی مدولاسیون همدوس، **detectedSymbolsIndex** را به دست آورید. توجه کنید که **realRxSymbols** در واقع خروجی ای است که می‌توان از آن برای رسم منظومه مدولاسیون 4FSK در چهار بعد استفاده نمود.

۲. محاسبه‌ی خطای سمبل: با مقایسه‌ی تعداد اختلاف‌های بردارهای **detectedSymbolsIndex** و **symbolIndex** و محاسبه نسبت اختلاف این دو بردار به تعداد کل سمبل‌ها، خطای سمبل را به دست آورده و آن را در متغیر **ser** قرار دهید.

۳. تبدیل سمبل به بیت و محاسبه‌ی خطای بیت: بیت‌های متناظر با سطرهای **detectedSymbolsIndex** را به دست آورید و آن را درون متغیر **detectedBits** قرار دهید. با مقایسه‌ی تعداد اختلاف ماتریس‌های **detectedBits** و **txBit** و محاسبه نسبت تعداد اختلاف این دو ماتریس به تعداد کل بیت‌ها، خطای بیت را به دست آورده و آن را در متغیر **ber** قرار دهید.

آزمایش ۵-۵: خواسته‌های کلی

۶. طیف سیگنال ارسالی و اثر Δf بر روی پهنای باند: طیف فرکانسی سیگنال ارسالی **txSamples** را برای مدولاسیون 2FSK رسم نمایید. در مورد پهنای باند و سطح باندهای جانبی آن توضیحاتی ارائه دهید. با تغییر Δf از $\frac{1}{2T}$ به $\frac{2}{T}$ پهنای باند سیگنال دریافتی هر حالت را به دست آورده و با یکدیگر مقایسه نمایید.

۷. رسم نمودار نرخ خطای بیت: با تغییر نسبت سیگنال به نویز بین 0 تا 10dB نرخ خطای بیت را برای مدولاسیون‌های 2FSK، 4FSK و 8FSK به دست آورده و در نمودارهایی جداگانه رسم نمایید و آن را با خروجی تابع **berawgn** مقایسه نمایید.

۸. اثر تأخیر در کانال:

در این جا فرض نمایید مدولاسیون 2FSK است و نسبت سیگنال به نویز (E_b/N_0) را از 0dB تا 10dB تغییر دهید.

گام ۱. اثر تأخیر در کانال بر روی نرخ خطای بیت و منظومه‌ی سیگنالی: تأخیر کانال را برابر با گرد شده‌ی 0.1، 0.5 و 0.8 برابر طول یک سمبل قرار دهید (یک عدد صحیح معادل تعداد نمونه‌های زمانی تأخیر یافته). نرخ خطای بیت را به دست آورده و با حالت بدون تأخیر مقایسه نمایید. برای نسبت سیگنال به نویز 40dB، منظومه سیگنالی حالت‌های فوق را نیز با یکدیگر مقایسه کرده و علت مشاهدات خود را توضیح دهید. (**isGray = 0**، **delayMode = 0**، **isHeader = 0** **channelPhaseOffset = 0**).

گام ۲. اضافه کردن هدر: برای یافتن نقطه‌ی شروع درست داده‌ها، تعدادی داده‌ی مشخص به عنوان هدر در ابتدای رشته بیت‌های ارسالی قرار دهید (دقت داشته باشید که طول **txBit** نباید افزایش یابد). بیت‌های هدر باید به صورت زیر باشد. این رشته بیت خواص خودهمبستگی خوبی دارد.

`headerBits = repmat(reshape(de2bi(hex2dec(['1C6387FF5DA4FA325C895958DC5'])))', [], 1), 1, k);`
سپس نمونه‌های زمانی مربوط به هدر را از بردار **txSamples** جدا کرده و در متغیر **headerSamples** ذخیره نمایید. با استفاده از متغیر **isHeader**، برنامه را برای حالتی که هدر وجود ندارد مدیریت کنید.

گام ۳. تخمین زمان شروع بسته و جبران اثر تأخیر کانال: با استفاده از همبستگی بردار **rxSamplesPlusNoise** و نمونه‌های زمانی هدر **headerSamples**، نقطه‌ی شروع بسته‌ی داده را تخمین بزنید. با مقایسه‌ی نرخ خطای بیت حالت بدون تأخیر و حالتی که نقطه‌ی شروع داده‌ها تخمین زده می‌شود، از عملکرد برنامه‌ی خود اطمینان حاصل نمایید. توجه کنید که این قسمت فقط زمانی اجرا می‌شود که **isHeader = 1** باشد.

۹. اعمال فاز در کانال و تخمین فاز:

در این جا فرض نمایید مدولاسیون 2FSK است و نسبت سیگنال به نویز (E_b/N_0) را از 0dB تا 10dB تغییر دهید.

گام ۱. اثر فاز اضافی کانال بر نرخ خطای بیت: اختلاف فاز کانال را برابر ۳۰ درجه قرار دهید. ابتدا نمودار نرخ خطای بیت را به دست آورده و با حالت بدون اختلاف فاز مقایسه نمایید. برای نسبت سیگنال به نویز 40dB، منظومه سیگنالی فاز کانال صفر درجه را با ۳۰ درجه مقایسه نمایید. علت مشاهدات خود را توضیح دهید (**delayMode = 0**، **isHeader = 0** و **channelDelayInSample = 0**).

گام ۲. تخمین فاز اضافی کانال: اگر به ابتدای بسته‌ی ارسالی هدر بخش قبل اضافه شود، با استفاده از همبستگی بردار **rxSamplesPlusNoise** و نمونه‌های زمانی هدر **headerSamples**، فازی را که کانال به سیگنال اضافه کرده است را به دست آورید. توجه کنید که این قسمت فقط زمانی اجرا می‌شود که **isHeader = 1** باشد. راهنمایی: به فاز در قله‌ی خروجی همبستگی دقت نمایید.

گام ۳. جبران سازی فاز کانال: تلاش کنید با اعمال ضریب مناسب اثر فاز اضافی کانال را جبران نمایید.

۱۰. جبران سازی: تابع `rxSyncAndCompensate` را طوری بنویسید که عملیات انجام شده برای جبران سازی تأخیر و فاز را به صورت منسجم در قالب یک تابع انجام دهد. ورودی های این تابع نمونه های زمانی هدر (`headerSamples`)، نمونه های نویزی سیگنال دریافتی (`rxSamplesPlusNoise`)، اندازه ی قله ی خودهمبستگی نمونه های زمانی هدر (`maxRx`) و نوع جبران سازی (`compensateMode`) بوده و خروجی آن نمونه های دریافتی پس از جبران سازی (`rxSignalSynch`) است.

آزمایش ۵-۶: پیاده سازی سخت افزاری گیرنده و فرستنده ی FSK

در این بخش `isHardware = 1` و `nPacketSymbols = 1000` می باشد.

۱. ارسال نمونه ها با استفاده از ADALM-PLUTO: بردار `txSamples` مربوط به مدولاسیون 2FSK (هدر اضافه شود) را با استفاده از دستور `transmitRepeat` به مدت زمان `dataTransferDuration` ثانیه به صورت پی در پی در فضا ارسال نمایید. این دستور داده ها را از طریق USB به رادیو نرم افزار ارسال می نماید و در حافظه ی سخت افزار ذخیره می نماید. فرستنده داده ها را از حافظه ی رادیو نرم افزار قرائت کرده و مدام ارسال می نماید. برای این منظور ابتدا می بایست پس از پیکربندی رادیو نرم افزار یک شیء فرستنده ایجاد نموده و فرکانس مرکزی آن را بر روی 2400MHz تنظیم کرده و بهره ی آن را برابر با 0dB تنظیم نمایید.

۲. دریافت سیگنال با استفاده از ADALM-PLUTO: ابتدا شیء مربوط به گیرنده را تعریف نمایید. فرکانس مرکزی آن را بر روی 2400MHz تنظیم کرده و بهره ی آن را به صورت Manual و برابر 20dB تنظیم نمایید. به علت اطمینان از دریافت یک بسته ی کامل، تعداد نمونه های زمانی دریافتی را برابر با دو برابر تعداد نمونه های زمانی بسته در نظر بگیرید. به منظور دریافت داده، شیء گیرنده را فراخوانی نمایید و حاصل را درون متغیر `rxSamplesPlusNoise` قرار دهید.

۳. طیف سیگنال دریافتی: طیف فرکانسی سیگنال `rxSamplesPlusNoise` دریافتی از ADALM-PLUTO را برای مدولاسیون 2FSK رسم نمایید. در مورد پهنای باند و سطح باندهای جانبی آن توضیحاتی ارائه دهید.

۴. دمدولاسیون و مشاهده ی منظومه ی سیگنالی:

- گام ۱. مشابه قبل عمل دمدولاسیون را انجام داده و منظومه ی سیگنالی مربوط به `rxSymbols` را رسم نمایید. منظومه ی سیگنالی باید بتواند به صورت به لحظه به روز شود.
- گام ۲. علت تفاوت منظومه ی سیگنالی در هنگام کار با سخت افزار را با حالت شبیه سازی را با مشاهده ی تفاوت شکل موج سیگنال دریافتی در حالت کار با سخت افزار و شبیه سازی توضیح دهید.
- گام ۳. با تغییر ویژگی های زیر برای سخت افزار مجدد مشاهدات خود را بیان نمایید. (توضیح بیشتر با جستجوی عبارت DC Offset Tracking در راهنمای نرم افزار MATLAB)

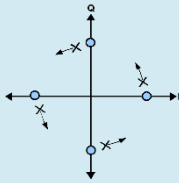
```
rx.ShowAdvancedProperties = true;
rx.EnablebitAverageEnergyasbitAverageEnergyandDCCorrection = false;
```

۵. جبران سازی دامنه، فاز و تأخیر و مشاهده ی منظومه ی سیگنالی: با یافتن ابتدای بسته ها و جبران فاز، دامنه و تأخیر، منظومه ی سیگنالی مربوط به `rxSymbols` را رسم نمایید. احتمال خطای بیت را نیز در این حالت محاسبه نمایید.

۶. اثر Δf بر روی پهنای باند ارسالی: با تغییر Δf از $\frac{1}{2T}$ به $\frac{2}{T}$ پهنای باند سیگنال دریافتی هر حالت را به دست آورده و با یکدیگر مقایسه نمایید.



- [1] J. G. Proakis and M. Salehi, *Digital Communications*. Boston: McGraw-Hill, 2008.
- [2] R. W. Stewart, K. Barlee, L. Crockett, and D. Atkinson, *Software Defined Radio using MATLAB & Simulink and the RTL-SDR*. 2015.



آزمایش ششم

آشکارسازی ناهمدوس

اهداف آزمایش

در طول آزمایش‌های قبل تمرکز بر روی آشکارسازی همدوس بوده است. در صورت وجود تأخیر کانال و تغییر فاز سیگنال دریافتی، ابتدا این فاز جبران و سپس آشکارسازی صورت می‌پذیرفت. در این آزمایش با مدولاسیون‌هایی آشنا می‌شویم که با وجود جبران نکردن فاز می‌توان عمل آشکارسازی را انجام داد که به این آشکارسازی، آشکارسازی ناهمدوس گفته می‌شود.

در انتهای این آزمایش دانشجو می‌بایست:

-  با مفهوم آشکارسازی ناهمدوس آشنا شود.
-  آشکارسازی ناهمدوس مدولاسیون FSK را انجام دهد.
-  با نحوه‌ی تولید و آشکارسازی مدولاسیون DBPSK آشنا شود.
-  با شبیه‌سازی کانال با فاز تصادفی آشنا باشد.
-  این نوع آشکارسازی را با سخت‌افزار پیاده‌سازی نماید.

ابزارهای مورد نیاز

-  رایانه
-  نرم‌افزار MATLAB R2020b
-  رادیو نرم‌افزار ADALM-PLUTO
-  کابل کوآکسیال
-  آنتن



آشکارسازی ناهمدوس

سیگنال پس از عبور از کانال دچار تأخیر زمانی می‌شود. این تأخیر زمانی باعث ایجاد ابهام فاز در سیگنال دریافتی می‌گردد. به عنوان مثال، سیگنال مربوط به مدولاسیون BPSK به صورت زیر است:

$$S_m(t) = \pm g(t) \cos(2\pi f_0 t) \equiv A_m g(t) \cos(2\pi f_0 t), \quad A_m \in \{\pm 1\}$$

در این جا بُعد فضا برابر با ۱ است و پایه‌ی این فضا به صورت زیر می‌باشد:

$$\Phi(t) = \sqrt{\frac{2}{E_g}} g(t) \cos(2\pi f_0 t)$$

پس از اضافه شدن نویز، سیگنال دریافتی در گیرنده به صورت زیر خواهد بود:

$$r(t) = S_m(t) + n(t) = \pm g(t) \cos(2\pi f_0 t) + n(t)$$

بنابراین برای آشکارسازی کافی است که در هر بازه‌ی T_s ضرب داخلی $\phi(t)$ و $r(t)$ را محاسبه شده و با توجه به علامت آن

تصمیم‌گیری انجام شود. این نحوه‌ی تصمیم‌گیری در ادامه آمده است:

$$\hat{A}_m = \text{sgn}(\langle r(t), \Phi(t) \rangle) = \text{sgn}\left(\int r(t)\Phi(t)dt\right)$$

در بسیاری از موارد، سیگنال ارسالی پس از عبور از کانال دچار تأخیر زمانی می‌شود و سیگنال دریافتی در گیرنده به صورت زیر

خواهد بود:

$$r(t) = S_m(t - \tau) + n(t) = \pm g(t) \cos(2\pi f_0 t + \phi) + n(t)$$

در رابطه‌ی فوق $\phi = -2\pi f_0 \tau$ می‌باشد.

معمولاً این تأخیر، در مقایسه با پهنای پالس ارسالی ناچیز است. اما تأثیر این تأخیر بر روی فاز کاملاً قابل ملاحظه است. در صورتی

که از تأثیر این تأخیر زمانی بر روی $g(t)$ صرف نظر کنیم، سیگنال پایه‌ی فضا به صورت زیر خواهد بود:

$$\Phi(t) = \sqrt{\frac{2}{E_g}} g(t) \cos(2\pi f_0 t + \phi)$$

بنابراین برای تشکیل بردار پایه باید مقدار ϕ را داشته باشیم که نیاز به تخمین τ دارد. فرض کنید که مقدار $\tau = 10^{-5}$ s باشد.

فرآیند تخمین زدن همواره با خطا همراه است و این خطا را به عنوان مثال ۱٪ در نظر می‌گیریم. بنابراین مقدار τ تخمین زده شده برابر

است با:

$$\hat{\tau} = 10^{-5} \pm 10^{-7} \text{ s}$$

فرکانس حامل (f_0) در سامانه‌های مختلف، مقادیر متفاوتی دارد. به عنوان مثال در شبکه‌ی سلولی مقدار فرکانس حامل از مرتبه‌ی

$10^9 \text{ Hz} \equiv 1 \text{ GHz}$ می‌باشد. بدین ترتیب ϕ تخمین زده شده در صورت زیر خواهد بود:

$$\hat{\phi} = -2\pi f_0 \hat{\tau} = -2\pi f_0 (10^{-5} \pm 10^{-7}) = \phi \pm 2\pi \times 10^2$$

همان گونه که ملاحظه می‌شود، با وجود خطای بسیار کوچک در تخمین، خطای قابل ملاحظه‌ای در ϕ حاصل خواهد شد.

در چنین مواردی بهتر است به جای تخمین پارامتر مبهم، آن را به صورت یک پارامتر غیر یقینی وارد محاسبات کرده و آن را در

تصمیم‌گیری وارد نماییم. برای مثال می‌توان ϕ را به عنوان یک متغیر تصادفی با توزیع یکنواخت بین $[0, 2\pi)$ در نظر گرفت و تصمیم‌گیری

بهبهینه را انجام داد. به چنین فرآیندی آشکارسازی ناهمدوس گفته می‌شود.

فرض کنید که مجموعه سیگنال‌های ارسالی به صورت $S_m(t) = g_m(t) \cos(2\pi f_0 t)$ باشد. $S_m(t, \phi)$ را به صورت زیر تعریف

می‌کنیم:

$$S_m(t, \phi) \triangleq g_m(t) \cos(2\pi f_0 t + \phi)$$

معادل باند پایه سیگنال $S_m(t)$ را با $S_{\ell,m}(t)$ نشان می‌دهیم. بنابراین:

$$S_m(t) = \Re\{S_{\ell,m}(t)e^{j2\pi f_0 t}\} \leftrightarrow S_m(t, \phi) = \Re\{S_{\ell,m}(t)e^{j2\pi f_0 t + j\phi}\}$$

سیگنال دریافتی معادل باند پایه برای این سیستم به صورت زیر است:

$$r_l(t) = S_{\ell,m}(t)e^{j\phi} + n_l(t)$$

می‌توان اثبات کرد با فرض اینکه ϕ یک متغیر تصادفی با توزیع یکنواخت بین $[0, 2\pi)$ است، تصمیم‌گیری بهینه برای اینکه احتمال

خطا مینیمم باشد به صورت زیر است:



$$\hat{m} = \underset{m=1, \dots, M}{\operatorname{argmax}} \left\{ \pi_m e^{\frac{-E_{\ell, m}}{2N_0}} I_0 \left(\frac{|\langle r_{\ell}, \underline{S}_{\ell, m} \rangle|}{N_0} \right) \right\}$$

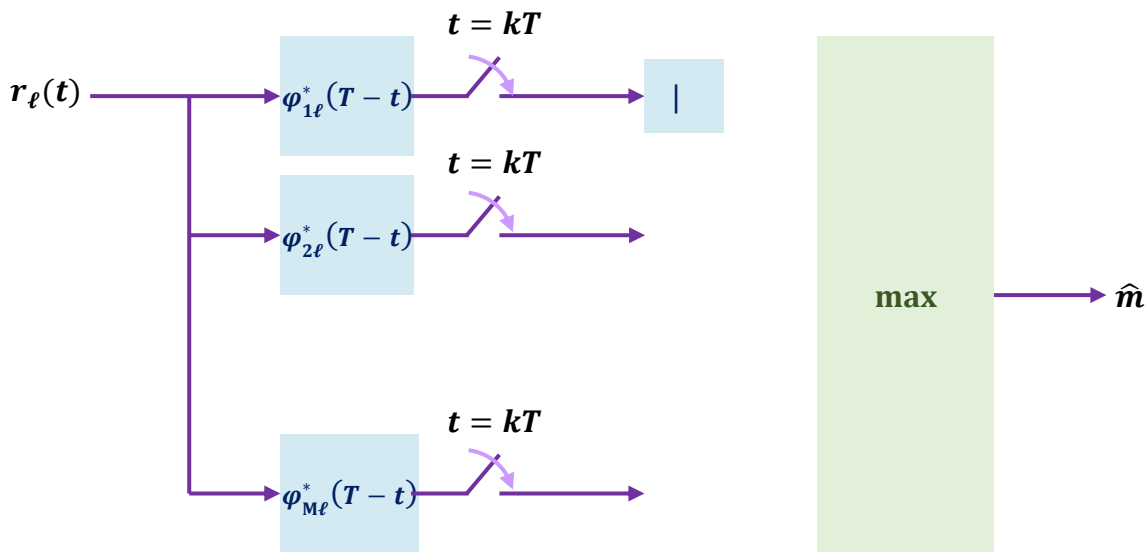
در رابطه‌ی فوق r_{ℓ} معادل برداری سیگنال دریافتی در باند پایه، $\underline{S}_{\ell, m}$ معادل برداری سیگنال ارسالی m ام در باند پایه و π_m احتمال ارسال سمبل m ام می‌باشد. $I_0(x)$ نیز تابع بسل اصلاح شده مرتبه 0 است. این تابع نسبت به x متقارن است و به ازای مقادیر مثبت x صعودی نیز می‌باشد. عملگر $\langle \cdot \rangle$ معادل ضرب داخلی دو بردار است که به صورت زیر تعریف می‌شود:

$$\underline{X} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix}, \underline{Y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} \rightarrow \langle \underline{X}, \underline{Y} \rangle = \underline{Y}^H \underline{X}$$

بدین ترتیب اگر احتمال ارسال همه سمبل‌ها برابر و انرژی سیگنال ارسالی متناظر با هر کدام از آن‌ها یکسان باشد، عبارت فوق به صورت زیر ساده خواهد شد:

$$\hat{m} = \underset{m}{\operatorname{argmax}} (|\langle r_{\ell}, \underline{S}_{\ell, m} \rangle|)$$

گیرنده متناظر با این تصمیم‌گیری به صورت زیر خواهد بود:

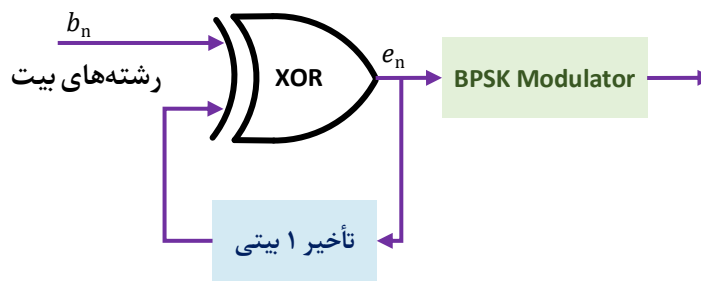


شکل ۱ شمای گیرنده‌ی ناهمدوس

مدولاسیون DPSK (Differential PSK)

سیگنال ارسالی در مدولاسیون PSK در حالت کلی به صورت $g(t) \cos \left(2\pi f_0 t + \frac{2\pi}{M} (m-1) \right)$ است. بنابراین تأخیر زمانی ایجاد شده به دلیل غیرایده‌آل بودن کانال، باعث می‌شود که سیگنال دریافتی به صورت $g(t) \cos \left(2\pi f_0 t + \frac{2\pi}{M} (m-1) + \phi \right)$ باشد. بنابراین نیاز است که آشکارسازی به صورت ناهمدوس صورت بگیرد. برای سادگی کار فرض می‌کنیم که مدولاسیون به ازای $M = 2$ (BPSK) انجام پذیرفته باشد. احتمال خطای آشکارسازی ناهمدوس برای مدولاسیون BPSK برابر $\frac{1}{2}$ است. این احتمال خطا نشان‌دهنده این واقعیت است که در صورت وقوع ابهام فاز در مدولاسیون BPSK، نمی‌توان در مورد سمبل ارسالی تصمیم‌گیری نمود و این مدولاسیون عملاً ناکارآمد خواهد شد.

برای مقابله با چنین اتفاقی، مدولاسیون DPSK (در حالت خاص DBPSK) پیشنهاد می‌شود. سمبل‌های ارسالی ما به صورت 0 و 1 (± 1) خواهد بود. در این مدولاسیون، به ازای وقوع هر 1، فاز سیگنال ارسالی π رادیان و به ازای وقوع هر 0، فاز سیگنال ارسالی 0 رادیان تغییر خواهد کرد. بدین ترتیب در گیرنده برای آشکارسازی می‌بایست اختلاف فاز سیگنال فعلی را با سیگنال قبل به دست آوریم و تصمیم‌گیری انجام دهیم. این کار باعث می‌شود که ابهام ϕ در فاز حذف شود و عملکرد مدولاسیون BPSK بهبود یابد. همان‌گونه که ملاحظه می‌شود، این مدولاسیون یک مدولاسیون حافظه‌دار است که برای تصمیم‌گیری برای سمبل فعلی، به سمبل قبلی نیاز خواهد بود. در عمل می‌توان برای پیاده‌سازی مدولاتور این مدولاسیون به صورت زیر عمل کرد.

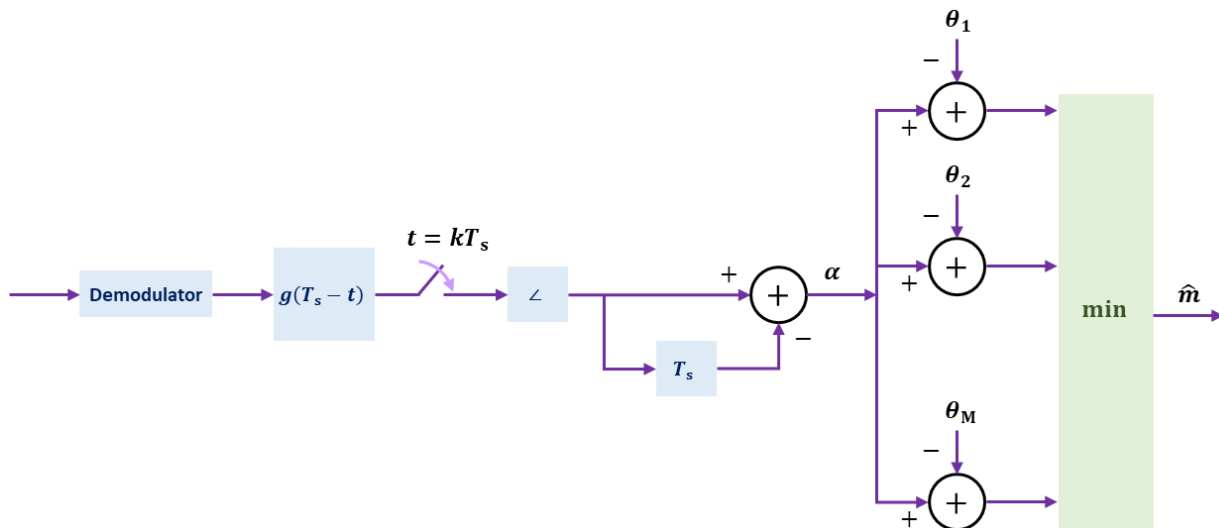


شکل ۲ شمای مدولاسیون DPSK

بنابراین برای پیاده‌سازی مدولاتور DBPSK، ابتدا رشته بیت ورودی (b_n) را به صورت زیر کدگذاری می‌کنیم.

$$e_k = e_{k-1} \oplus b_k$$

سپس با توجه به رشته بیت تولید شده، به ازای هر 0، فاز 0 و هر 1، فاز π را در نظر گرفت. دمدولاتور DPSK نیز به صورت زیر می‌باشد.



شکل ۴ شمای گیرنده‌ی مدولاسیون DPSK

که برای حالت خاص $m = 2$ می‌توان به صورت زیر تصمیم‌گیری نمود

$$\Re\{r_{\ell,k} r_{\ell,k-1}^*\} > 0 \rightarrow \hat{b}_k = 0$$

$$\Re\{r_{\ell,k} r_{\ell,k-1}^*\} < 0 \rightarrow \hat{b}_k = 1$$

$r_{\ell,k}$ معادل برداری سیگنال پایه دریافتی در k امین بازه‌ی سمبل (T_s) است.

شرح آزمایش

آزمایش ۱-۶: مقداردهی‌های اولیه

در این آزمایش بنا داریم تعدادی بسته‌ی داده را برای ارسال از طریق مدولاسیون FSK ناهمدوس و DBPSK آماده نماییم. از آن جا که در این آزمایش بناست این فرستنده با استفاده از رادیو نرم افزار ADALM-PLUTO پیاده‌سازی شود، می‌بایست با استفاده از پارامترها و ملاحظات عملی، این کار انجام شود.

پارامترهایی که در این پیاده‌سازی اهمیت دارد شامل نرخ نمونه‌برداری (f_s)، تعداد نمونه‌های هر سمبل ($nSymbolSamples$)، تعداد سمبل‌های ارسالی در بسته‌ی داده ($nPacketSymbols$) و مدت زمان ارسال داده ($dataTransferDuration$) است. در این آزمایش مقدار پارامترهای مورد نیاز به صورت جدول ۱ می‌باشد.

جدول ۱ پارامترهای آزمایش ۶

پارامتر	f_s	$nSymbolSamples$	$nPacketSymbols$	$dataTransferDuration$
مقدار	10MHz	FSK: 64 DBPSK: 8	شبیه‌سازی: ۱۰۰۰۰۰ سخت‌افزار: ۱۰۰۰	۱۰۰ ثانیه

دقت نظر داشته باشید که در این آزمایش نیز تنها می‌خواهیم برنامه‌هایی که در آزمایش قبل نوشته شده است را تکمیل نماییم.

۱. تولید M-File مربوط به مقدار دهی اولیه: قالب کلی پارامترهای مورد نیاز در این آزمایش، مانند آزمایش قبل است. فایل `dclInitialize.m` باید پارامتری به نام `detectionMode` داشته باشد تا با استفاده از آن بتوان نوع آشکارسازی همدوس (`coherent`) یا ناهمدوس (`noncoherent`) را مشخص نمود. هم‌چنین از پرچم `channelPhaseMode` برای وجود یا عدم وجود ابهام فاز استفاده خواهد شد. این فایل را باید از برنامه اصلی فراخوانی نمایید.

آزمایش ۲-۶: پیاده‌سازی مدولاسیون FSK ناهمدوس

۱. شبیه‌سازی کانال با فاز تصادفی: در شبیه‌سازی کانال با فاز تصادفی، هر سمبل با یک فاز تصادفی جمع می‌شود و این بدان معناست که همه سمبل‌های مربوط به یک سمبل مشخص، افزایش فاز یکسانی را تجربه می‌کنند. در نتیجه برای شبیه‌سازی به ترتیب زیر عمل کنید:

گام ۱. به اندازه سمبل‌ها، فاز تصادفی بین ۰ و 2π تولید کرده و حاصل را در بردار `channelPhaseSymbols` قرار دهید.

گام ۲. بردار `channelPhase` را که طول آن برابر طول بردار `txSamples` است، طوری بسازید که هر المان آن از تکرار المان‌های `channelPhaseSymbols` به اندازه `nSymbolSamples` به دست آمده باشد. راهنمایی: از دستور `kron` می‌توان برای این کار استفاده نمود.

گام ۳. بردار `txSamplesDelayed` را در `exp(1i*channelPhase)` کرده و حاصل را در بردار `txSamples` قرار دهید.

۲. شبیه‌سازی مدولاسیون FSK ناهمدوس: برای پیاده‌سازی این مدولاسیون، ابتدا توابع `pulseModulation` و `pulseDemodulation` را برای حالت ناهمدوس توسعه داده و همانند FSK همدوس که در آزمایش قبل پیاده شد عمل نمایید. نمودار احتمال خطای بیت را برای مدولاسیون 4FSK و 2FSK به دست آورده و نتیجه‌ی را با دستور `berawgn` صحت‌سنجی نمایید. دقت نمایید در حالت آشکارسازی ناهمدوس باید حداقل فاصله‌ی فرکانسی $1/T_s$ باشد. آشکارساز این مدولاسیون نیز به صورت $\arg \max |r_{\ell} \cdot \underline{s}_{\ell, m}|$ می‌باشد.

۳. پیاده‌سازی سخت‌افزاری مدولاسیون FSK ناهمدوس: با فرض این که توان فرستنده‌ی ADALM-PLUTO برابر با 0dB و بهره‌ی گیرنده برابر با 20dB باشد و از آنتن به منظور ارتباط فرستنده و گیرنده استفاده شود، خطای آشکارسازی با استفاده از آشکارسازی ناهمدوس را برای مدولاسیون 2FSK محاسبه نمایید. هم‌چنین در این حالت منظومه‌ی سیگنالی این مدولاسیون را نیز رسم نمایید. در این جا از هدر تنها برای به دست آوردن ابتدای بسته‌ی ارسالی استفاده می‌شود و هیچ گونه جبران‌سازی فازی صورت نمی‌پذیرد (`compensateMode = 0 or 1`).

آزمایش ۳-۶: پیاده‌سازی مدولاسیون DBPSK

۱. تولید بیت: با استفاده از تابع `bitGenerator` تعدادی بیت ۰ و ۱ را متناسب با طول بسته‌ی داده و با در نظر گرفتن پارامترهای مدولاسیون DBPSK تولید نموده و آن را درون ماتریس `txBit` ذخیره نمایید. ماتریس `txBit` یک ماتریس با اندازه‌ی $nPacketSymbols \times 1$ می‌باشد.

۲. کد کردن بیت‌ها: با استفاده از شکل رشته بیت‌های e_n را تولید نمایید و آن را درون بردار `encodedTxBits` قرار دهید.

۳. نگاشت بیت به سمبل:

گام ۱. کدگذاری گری بیت‌های تولیدی: با وجود این که در این جا نیازی به کدگذاری گری نیست ولی به منظور کلی بودن برنامه این مرحله نیز انجام می‌شود. با استفاده از تابع `grayMatrixGenerator`، ماتریس مربوط به کدگذاری گری مدولاسیون DBPSK را تولید کنید و آن را در ماتریس `grayMatrix` ذخیره کنید. بردار جدیدی متناظر با ماتریس `encodedTxBits` و با نام `symbolIndex` تولید نمایید و نگاشتی یک‌به‌یک بین سطرها‌ی ماتریس `encodedTxBits` و ماتریس `grayMatrix` برقرار کنید. به عبارتی سطر i ام بردار `symbolIndex`، شماره‌ی سطری از `grayMatrix` را نشان می‌دهد که برابر با سطر i ام ماتریس `encodedTxBits` باشد.

گام ۲. تولید سمبل‌های ارسالی: با استفاده از تابع `constellation`، تمامی سمبل‌های ارسالی مدولاسیون DBPSK را تولید کرده و سمبل‌های متناظر با هر سطر بردار `symbolIndex` را تولید کنید. این سمبل‌ها را در بردار `symbolArray` ذخیره نمایید.

۴. شکل‌دهی پالس ارسالی: با استفاده از تابع `pulseModulation` نمونه‌های ارسالی مربوط به یک بسته‌ی داده را آماده‌ی ارسال کرده و حاصل را در بردار `txSamples` ذخیره نمایید. تابع شکل‌دهی پالس را مثلی در نظر بگیرید.

آزمایش ۴-۶: مدل‌سازی کانال

۱. افزودن تأخیر در کانال: برای شبیه‌سازی تأخیر در کانال به اندازه‌ی پارامتر `channelDelayInSample` به ابتدای بردار `txSamples` صفر اضافه نمایید. حاصل را درون بردار `txSamplesDelayed` قرار دهید. دقت نمایید که طول بردار افزایش می‌باید و می‌بایست در ادامه‌ی برنامه اثر این افزایش لحاظ شود. در این جا فعلاً مقدار تأخیر برابر با صفر قرار دهید.

۲. اعمال ابهام فاز کانال: مدولاسیون DPSK به تغییرات شدید فاز حساس است و بنابراین باید ابهام فاز ایجاد شده خیلی زیاد نباشد. ابهام فاز ایجاد شده برای سمبل‌ها را به صورت زیر مدل می‌کنیم

`channelPhase = 0:phase_stepphaseStep:(nPacketSymbols-1)*phase_stepphaseStep`
پارامتر `phase_stepphaseStep`، مقدار ابهام فاز ایجاد شده برای یک سمبل نسبت به سمبل قبلی را مشخص می‌کند. حال با استفاده از دستور `kron`، فاز تولید شده را برای کل نمونه‌های هر سمبل تکرار کنید. با ضرب کردن عبارت `exp(1j*channelPhase)` در `txSamplesDelayed`، ابهام فاز ایجاد شده را به سینگال ارسالی اضافه نمایید و حاصل را در `txSamples` ذخیره نمایید. در این جا `phase_stepphaseStep` را برابر با ۱۰ درجه قرار دهید.

۳. شبیه‌سازی کانال با نویز سفید گاوسی

- a. تعیین واریانس نویز بر اساس نسبت سیگنال به نویز (E_b/N_0): ابتدا بر اساس خروجی تابع **constellation**، مقدار متوسط انرژی سمبل را به دست آورید (**symbolAverageEnergy**). حال با استفاده از متوسط انرژی سمبل، مقدار انرژی متوسط بیت را به دست آورده و برابر متغیر **bitAverageEnergy** قرار دهید. مقدار E_b/N_0 را برابر با 10dB در نظر بگیرید (به عبارتی پارامترهای **snrMin** و **snrMax** برابر 10 تنظیم می شوند). سپس واریانس نویز را بر اساس نسبت E_b/N_0 به دست آورید و آن را درون متغیر **noiseVariance** قرار دهید.
- b. افزودن نویز به سیگنال: با استفاده از تابع **randn** یک بردار نویز مختلط با واریانس **noiseVariance** و ابعاد برابر با **txSamples** تولید نمایید و آن را **noiseSamples** نامگذاری نمایید. سپس این بردار را با بردار **txSamples** جمع نموده و حاصل را **txSamples_plusNoise** بنامید.

آزمایش ۵-۶: پیاده سازی گیرنده ی مدولاسیون DBPSK

- در ادامه ی فایل آزمایش قبل می خواهیم گیرنده ی مدولاسیون DBPSK را پیاده سازی نماییم.
۱. دمدولاسیون یا آشکارسازی پالس: نمونه های ارسال شده به همراه نویز (**txSamplesPlusNoise**) را به عنوان نمونه های دریافتی درون بردار **rxSamples** قرار دهید. با استفاده از تابع **pulseDemodulation** با ورودی های نمونه های سیگنال دریافتی (**rxSamples**)، نام مدولاسیون (**modulation**)، مرتبه ی مدولاسیون (**M**)، نرخ نمونه برداری (**fs**)، تعداد نمونه های هر سمبل (**nSymbolSamples**)، نام تابع شکل دهنده ی پالس (**pulseName**)، روش تشخیص قدرت سیگنال دریافتی در راستای سیگنال های پایه (**mode**) و شیوه ی آشکارسازی (**detectionMode**)، سمبل های دریافتی (**rxSymbols**) و اندیس سمبل های آشکار شده (**detectedSymbolsIndex**) را به دست آورید.
 ۲. محاسبه ی خطای سمبل: با مقایسه ی تعداد اختلاف های بردارهای **detectedSymbolsIndex** و **symbolIndex** و محاسبه نسبت اختلاف این دو بردار به تعداد کل سمبل ها، خطای سمبل را به دست آورده و آن را در متغیر **ser** قرار دهید.
 ۳. تبدیل سمبل و بیت و تبدیل از کدگذاری گری به کدگذاری باینری: بیت های متناظر با سطرهای **detectedSymbolsIndex** را به دست آورید و آن را درون متغیر **detectedBitsGray** قرار دهید. با وجود این که در این جا عکس تبدیل گری مطرح نیست به منظور کلی ماندن برنامه بیت های گری را به کدگذاری باینری تبدیل نماید و آن را درون بردار **detectedBits** قرار دهید.
 ۴. محاسبه ی خطای بیت: با مقایسه ی تعداد اختلاف ماتریس های **detectedBits** و **txBit** و محاسبه نسبت تعداد اختلاف این دو ماتریس به تعداد کل بیت ها، خطای بیت را به دست آورده و آن را در متغیر **ber** قرار دهید.

آزمایش ۶-۶: خواسته های کلی

۱. رسم نمودار نرخ خطای بیت: با تغییر نسبت سیگنال به نویز بین 0 تا 10dB نرخ خطای بیت را برای مدولاسیون های DBPSK به دست آورده و در یک نمودار رسم نمایید و آن را با خروجی تابع **berawgn** مقایسه نمایید. این کار را برای شکل موج مثلثی انجام دهید.
۲. اثر تغییرات فاز بر روی عملکرد: با تغییر پارامتر **phaseStep**، تأثیر تغییرات فاز را بر روی عملکرد این سامانه ارزیابی نمایید.

آزمایش ۶-۷: پیاده سازی سخت افزاری مدولاسیون DBPSK

۱. ارسال نمونه ها با استفاده از ADALM-PLUTO: بردار **txSamples** مربوط به مدولاسیون DBPSK (هذر اضافه شود) را با استفاده از دستور **transmitRepeat** به مدت زمان **dataTransferDuration** ثانیه به صورت پی در پی در فضا ارسال نمایید. این دستور داده ها را از طریق USB به رادیو نرم افزار ارسال می نماید و در حافظه ی سخت افزار ذخیره می نماید.

فرستنده داده‌ها را از حافظه‌ی رادیونرم‌افزار قرائت کرده و مدام ارسال می‌نماید. برای این منظور ابتدا می‌بایست پس از پیکربندی رادیو نرم‌افزار یک شیء فرستنده ایجاد نموده و فرکانس مرکزی آن را بر روی 2400MHz تنظیم کرده و بهره‌ی آن را برابر با 0dB تنظیم نمایید.

۲. دریافت سیگنال با استفاده از ADALM-PLUTO: ابتدا شیء مربوط به گیرنده را تعریف نمایید. فرکانس مرکزی آن را بر روی 2400MHz تنظیم کرده و بهره‌ی آن را به صورت Manual و برابر 20dB تنظیم نمایید. به علت اطمینان از دریافت یک بسته‌ی کامل، تعداد نمونه‌های زمانی دریافتی را برابر با دو برابر تعداد نمونه‌های زمانی بسته در نظر بگیرید. به منظور دریافت داده، شیء گیرنده را فراخوانی نمایید و حاصل را درون متغیر **rxSamples** قرار دهید.

۳. دمدولاسیون و مشاهده‌ی منظومه‌ی سیگنالی: مشابه قبل عمل دمدولاسیون را انجام داده و منظومه‌ی سیگنالی مربوط به **rxSymbols** را رسم نمایید و مشاهدات خود را یادداشت نمایید. منظومه‌ی سیگنالی باید بتواند به صورت به‌لحظه به‌روز شود.



- [3] [1] R. W. Stewart, K. Barlee, L. Crockett, and D. Atkinson, *Software Defined Radio using MATLAB & Simulink and the RTL-SDR*. 2015.



آزمایش هفتم

انتقال دیجیتال از درون کانال باند محدود AWGN

اهداف آزمایش

در طول آزمایش‌های قبل فرض بر آن بود تمرکز بر روی آشکارسازی همدوس بوده است. در صورت وجود تأخیر کانال و تغییر فاز سیگنال دریافتی، ابتدا این فاز جبران و سپس آشکارسازی صورت می‌پذیرفت. در این آزمایش با مدولاسیون‌هایی آشنا می‌شویم که با وجود جبران نکردن فاز می‌توان عمل آشکارسازی را انجام داد که به این آشکارسازی، آشکارسازی ناهمدوس گفته می‌شود.

در انتهای این آزمایش دانشجو می‌بایست:

-  با مفهوم آشکارسازی ناهمدوس آشنا شود.
-  آشکارسازی ناهمدوس مدولاسیون FSK را انجام دهد.
-  با نحوه‌ی تولید و آشکارسازی مدولاسیون DBPSK آشنا شود.
-  با شبیه‌سازی کانال با فاز تصادفی آشنا باشد.
-  این نوع آشکارسازی را با سخت‌افزار پیاده‌سازی نماید.

ابزارهای مورد نیاز

-  رایانه
-  نرم‌افزار MATLAB R2020b
-  رادیو نرم‌افزار ADALM-PLUTO
-  کابل کوکسیال
-  آنتن



تئوری

کانال باند محدود

در آزمایش‌های قبل کانال مورد استفاده دارای باند فرکانسی نامحدود فرض می‌شد. در واقع مدل کانال مورد استفاده در حوزه‌ی زمان به صورت تابع ضربه بود. در عمل کانال‌های مخابراتی دارای پاسخ زمانی غیرایده‌آل است و باعث ایجاد تداخل بین سمبلی می‌شود. تداخل بین سمبلی، شکلی از اعوجاج سیگنال است که هر سمبل با چندین سمبل دیگر تداخل پیدا می‌کند. این امر، یک اتفاق ناخواسته است که سمبل‌های قبلی به دلیل پهن شدن در حوزه‌ی زمان، هم‌چون نویز برای سمبل فعلی ظاهر می‌شوند و قابلیت اطمینان سیستم‌های مخابراتی را کاهش می‌دهند. وجود تداخل بین سمبلی در سیستم باعث ایجاد خطا در تصمیم‌گیری می‌شود. بنابراین لازم است که فیلترهای فرستنده و گیرنده را به گونه‌ای طراحی کنیم که تا حد ممکن از خطاهای ایجاد شده جلوگیری شود.

دسته‌ی مهمی از کانال‌ها غیرایده‌آل، کانال‌های باند محدود هستند که پاسخ فرکانسی آنها برای فرکانس‌های مشخصی برابر با صفر است. عبور سیگنال از این کانال‌ها منجر به حذف فرکانس‌های خارج از باند گذر کانال می‌شود. از طرف دیگر، ممکن است که فرکانس‌هایی که در محدوده‌ی باند گذر کانال هستند نیز تضعیف شوند.

کانال‌های باند محدود هم در مخابرات سیمی و هم در مخابرات بی‌سیم وجود دارند. در بسیاری از موارد، این محدود شدن باند می‌تواند عمدی باشد. به عنوان مثال اگر بنا باشد چندین کاربر به صورت همزمان از یک محیط برای انتقال داده استفاده کنند، می‌توان به عنوان نمونه باند فرکانسی کاربران را از یکدیگر ایزوله نمود. محدود بودن باند می‌تواند به دلیل مشخصات فیزیکی محیط انتقال نیز باشد. به عنوان مثال ممکن است که یک کابل دارای فرکانس قطع مشخصی باشد و اجازه‌ی ارسال سیگنال در فرکانس‌های بالاتر را ندهد. معمولاً سیستم‌های مخابراتی که دارای کانال‌های باند محدود هستند از شکل‌دهی پالس برای جلوگیری از ایجاد تداخل استفاده می‌کنند.

الگوی چشم (Eye Pattern)

خروجی فیلتر ارسالی در سامانه‌ی مخابراتی با مدولاسیون PAM، PSK یا QAM را می‌توان به صورت زیر بیان نمود.

$$v(t) = \sum_{n=-\infty}^{+\infty} a_n g_T(t - nT)$$

هم‌چنین خروجی کانال که در واقع سیگنال دریافتی در دمدولاتور است نیز می‌توان به صورت زیر بیان کرد.

$$r(t) = \sum_{n=-\infty}^{+\infty} a_n h(t - nT) + n(t)$$

در رابطه‌ی فوق $h(t) = c(t) * g_T(t)$ است که در آن $c(t)$ پاسخ ضربه‌ی کانال، $g_T(t)$ پاسخ ضربه‌ی فیلتر ارسالی و $n(t)$ نیز

یک تابع نمونه از یک فرآیند نویز گاوسی سفید جمع شونده می‌باشد.

برای طراحی یک فیلتر ارسالی باند محدود ابتدا طراحی در شرایطی که هیچ اعوجاج کانالی وجود ندارد صورت می‌گیرد. سپس

اثر اعوجاج کانال مورد بررسی قرار می‌گیرد. از آن‌جا که $H(f) = C(f)G_T(f)$ شرط انتقال بدون اعوجاج آن است که پاسخ فرکانسی $C(f)$

کانال دارای دامنه‌ی ثابت و فازی خطی در کل پهنای باند سیگنال ارسالی باشد. به عبارتی به صورت رابطه‌ی زیر باشد.

$$C(f) = \begin{cases} C_0 e^{-j2\pi f t_0} & |f| \leq W \\ 0 & |f| > W \end{cases}$$

در عبارت فوق W پهنای باند در دسترس کانال، t_0 یک تأخیر دلخواه محدود و C_0 نیز یک ضریب بهره‌ی ثابت است. برای سادگی

می‌توان فرض نمود $C_0 = 1$ و $t_0 = 0$ است. در شرایطی که کانال بدون اعوجاج است برای $|f| \leq W$ خواهیم داشت $H(f) = G_T(f)$ و

برای $|f| > W$ صفر خواهد بود. در نتیجه فیلتر منطبق دارای پاسخ فرکانسی $H^*(f) = G_T^*(f)$ بوده و خروجی آن در زمان‌های

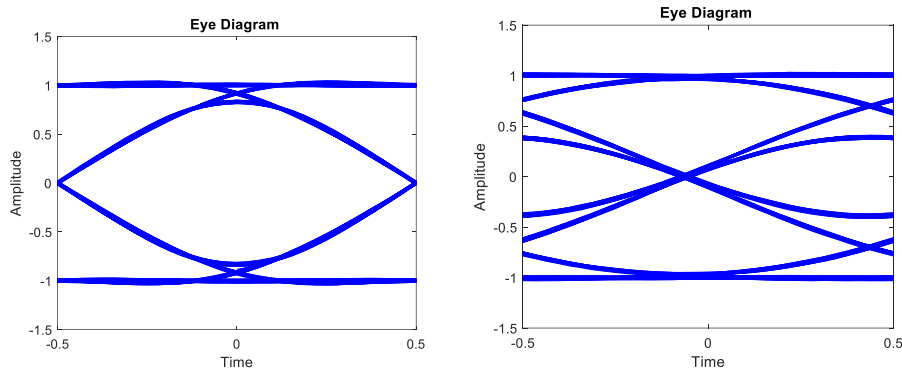
نمونه‌برداری متناوب $t = mT$ به صورت زیر است.

$$y(mT) = x(0)a_m + \sum_{n \neq m} a_n x(mT - nT) + v(mT) \equiv y_m = x_0 a_m + \sum_{n \neq m} a_n x_{m-n} + v_m$$

در عبارت فوق $x(t) = g_T(t) * g_R(t)$ و $v(t)$ نیز خروجی فیلتر منطبق به فرآیند AWGN ورودی $n(t)$ می‌باشد.

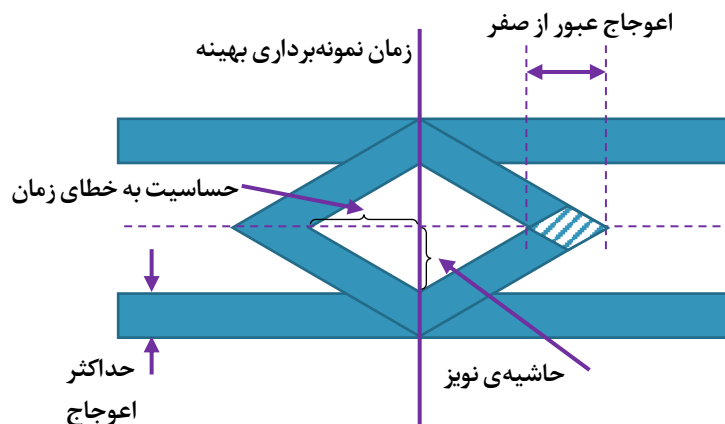


در رابطه‌ی فوق عبارت $\sum_{n \neq m} a_n x_{m-n}$ تداخل بین سمبلی (ISI) را بیان می‌کند. مقدار این تداخل بین سمبلی و نویز حاضر در سیگنال دریافتی را می‌توان بر روی یک نوسان نگار مشاهده نمود. برای این منظور می‌بایست سیگنال دریافتی را بر روی محور عمودی نشان داد و نرخ جاروب محور افقی را نیز برابر با $1/T$ قرار داد. نمایش حاصل بر روی نوسان نگار را الگوی چشم گویند که علت آن شباهتی است که با چشم انسان دارد. نمونه‌هایی از دو الگوی چشم در شکل 5 نشان داده شده است.



شکل 5 نمونه‌های از نمودار چشمی

تداخل بین سمبلی باعث بسته شدن چشم می‌شود و به عبارتی منجر به کاهش حاشیه‌ای که نویز جمع‌شونده ایجاد خطا می‌کند می‌شود. شکل 6 تأثیر تداخل بین سمبلی را بر روی کاهش باز بودن چشم نشان می‌دهد. می‌بایست توجه کرد که تداخل بین سمبلی لحظات گذر از صفر را دچار اعوجاج کرده و باعث کاهش باز بودن چشم می‌شود. در نتیجه این سامانه به خطای همگام‌سازی حساسیت بیشتری داشته و حاشیه‌ی کمتری در برابر نویز جمع‌شونده نشان می‌دهد.



شکل 6 شمای اثر تداخل بین سمبلی بر روی مقدار بازی الگوی چشم

شرح آزمایش

آزمایش ۱-۷: مقداردهی‌های اولیه

در این آزمایش بنا داریم تعدادی بسته‌ی داده را برای ارسال از طریق مدولاسیون 2PAM آماده نماییم. از آن جا که در این آزمایش بناسست این فرستنده با استفاده از رادیونرم افزار ADALM-PLUTO پیاده‌سازی شود، می‌بایست با استفاده از پارامترها و ملاحظات عملی، این کار انجام شود.

پارامترهایی که در این پیاده‌سازی اهمیت دارد شامل نرخ نمونه‌برداری (f_s)، تعداد نمونه‌های هر سمبل ($nSymbolSamples$)، تعداد سمبل‌های ارسالی در بسته‌ی داده ($nPacketSymbols$) و مدت زمان ارسال داده ($dataTransferDuration$) می‌باشد. در این آزمایش مقدار پارامترهای مورد نیاز به صورت جدول می‌باشد.

جدول 1 پارامترهای آزمایش ۷

پارامتر	f_s	$nSymbolSamples$	$nPacketSymbols$	$dataTransferDuration$
مقدار	10MHz	32	شبیه‌سازی: ۱۰۰۰۰۰ سخت‌افزار: ۱۰۰۰	۱۰۰ ثانیه

دقت نظر داشته باشید که در این آزمایش نیز تنها می‌خواهیم برنامه‌هایی که در آزمایش قبل نوشته شده است را تکمیل نماییم.

آزمایش ۲-۷: پیاده‌سازی مدولاسیون 2PAM

۱. شبیه‌سازی مدولاسیون 2PAM: برای پیاده‌سازی این مدولاسیون، همانند آزمایش ۴ یا ۵ عمل نمایید. احتمال خطای بیت را برای نسبت $\frac{E_b}{N_0}$ برابر با 10dB به دست آورید. این عمل را برای دو شکل پالس مستطیلی و Root Raised Cosine ($\beta = 0.9$) و $spanInSymbol = 6$ انجام دهید. (حالت عملکردی دمدولاتور را بر روی فیلتر منطبق تنظیم نمایید.)

۲. رسم نمودار چشمی: با استفاده از دستور **eyediagram** نرم‌افزار MATLAB نمودار چشمی مدولاسیون 2PAM را برای دو شکل پالس گفته شده و در دو نسبت E_b/N_0 برابر با 10dB و 40dB رسم نمایید. پارامترهای بر روی شکل 6 را به دست آورید.

آزمایش ۳-۷: مدل سازی کانال بانداریک

۱. تولید یک فیلتر FIR: با استفاده از دستور **fir1** نرم‌افزار MATLAB یک فیلتر با پهنای باند 300kHz و با تعداد ۱۰۰ تپ تولید نمایید. پاسخ فرکانسی و پهنای باند نویز فیلتر را به دست آورید و با پهنای باند سیگنال ارسالی مقایسه نمایید.

۲. کانال باندمحدود: پس از اعمال تأخیر و اعمال ابهام فاز، سیگنال ارسالی حاصل را با استفاده از عمل کانولوشن از فیلتر FIR مرحله‌ی قبل عبور دهید. مجدد نمودار چشمی را در دو نسبت E_b/N_0 برابر با 10dB و 40dB رسم نمایید. احتمال خطای بیت را نیز به دست آورید. این عمل برای هر دو شکل پالس گفته شده انجام شود.

آزمایش ۴-۷: پیاده‌سازی کانال باندمحدود

۱. کار با ADALM-PLUTO: موارد گفته شده را با استفاده از رادیونرم‌افزار ADALM-PLUTO نیز انجام دهید.

آزمایش ۵-۷: خواسته‌های کلی

۱. اثر طول شکل پالس root raised cosine: با تغییر پارامتر **spanInSymbol** تأثیر آن را بر روی نمودار چشمی و خطای بیت مدولاسیون 2PAM با شکل پالس root raised cosine را مشاهده نمایید.



- [1] R. W. Stewart, K. Barlee, L. Crockett, and D. Atkinson, *Software Defined Radio using MATLAB & Simulink and the RTL-SDR*. 2015.

